

高效的在线强化学习微调无需保留离线数据

Zhiyuan Zhou^{*1}, Andy Peng^{*1}, Qiyang Li¹, Sergey Levine¹, Aviral Kumar²

¹UC Berkeley, ²Carnegie Mellon University (*Equal Contribution)

现代机器学习范式涉及在多样化数据上进行预训练，随后进行任务特定的微调。在强化学习中，这转化为在多样化的历史数据集上通过离线强化学习进行学习，随后利用交互数据进行快速的在线强化学习微调。大多数强化学习微调方法需要继续在离线数据上训练以保持稳定性和性能。然而，这是不可取的，因为在大规模数据集上训练多样化的离线数据既缓慢又昂贵，并且原则上由于对离线数据的约束或悲观态度，也应限制可能的性能提升。在本文中，我们表明，只要使用适当设计的在线强化学习方法对离线强化学习初始化进行微调，保留离线数据是不必要的。为了构建这种方法，我们首先分析在在线微调中保留离线数据的作用。我们发现，继续在离线数据上训练主要用于防止微调开始时价值函数的突然发散，这种发散是由离线数据与在线轨迹之间的分布不匹配引起的。这种发散通常导致遗忘和丧失离线预训练的好处。我们的方法，热启动强化学习，通过一个非常简单的想法来缓解预训练初始化的灾难性遗忘。热启动强化学习采用一个预热阶段，用来自预训练策略的极少量轨迹来启动在线强化学习运行，以进行快速在线强化学习。预热期间收集的数据弥合了分布不匹配，并有助于将离线 Q 函数“重新校准”到在线分布，使我们能够完全丢弃离线数据而不破坏在线强化学习微调的稳定性。我们表明，热启动强化学习能够在不保留任何离线数据的情况下进行微调，并且与现有算法相比，无论它们是否保留离线数据，都能更快地学习并获得更高的性能。

1. 引言

当今大规模学习的主流范式是先多样化的先验数据上预训练模型，然后在更窄的领域特定数据上微调，使其专门化于特定的下游任务 [8, 5, 10, 41, 59, 54, 63]。在学习决策策略的背景下，这一范式转化为通过离线强化学习（RL）在大量先前收集的静态经验上进行预训练，随后通过在线强化学习高效地微调这些初始化。通常，这种微调是通过在离线数据和自主在线数据的混合体上，继续使用完全相同的离线强化学习算法（例如，悲观 [29, 7] 算法或施加行为约束 [15, 28] 的算法）进行训练，并对离线强化学习算法本身进行少量修改 [37]。

尽管这一范式带来了有前景的结果 [28, 37]，但强化学习微调需要继续在离线数据上训练以保持稳定性和性能 ([60, 61]; 第 3 节)，这与机器学习中的标准做法不同。保留离线数据存在若干问题。首先，随着离线数据集规模和多样性的增长，在离线数据上持续进行在线训练变得低效且昂贵，这种计算需求甚至可能使实践者不愿使用在线强化学习进行微调。其次，保留离线数据的需求或许完全违背了离线强化学习预训练的初衷：近期结果 [51]（以及我们在第 3 节的实验所证实）表明，当前的微调方法无法有效利用若干强大的离线强化学习价值函数和/或策略初始化，这体现在将离线数据放入回放缓冲区后从头运行在线强化学习能取得更优的性能 [3]。这些问题使当前强化学习微调方法的有效性受到质疑。

在本文中，我们旨在理解并解决当前在线微调方法的上述不足，并构建一种不保留离线数据的在线强化学习方法。为了开发我们的方法，我们首先实证分析在当前在线强化学习微调中保留离线数据的重要性

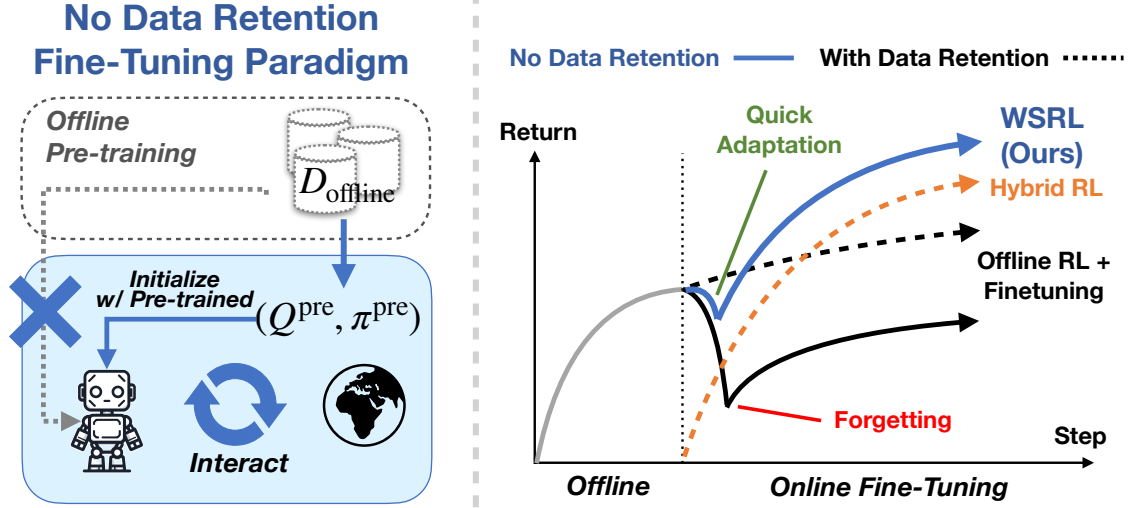


图 1：无数据保留微调专注于在线更新过程中不使用离线数据集的强化学习微调，反映了当今大规模机器学习中的常见范式。离线数据集仅用于通过离线强化学习预训练策略和 Q 函数以初始化微调，之后数据集被丢弃，智能体仅利用在线经验进行微调。当前方法在这种“无保留”设置下表现挣扎，并遗忘了预训练中学到的知识。我们的目标是开发一种即使不保留离线数据也能快速在线适应的微调方法。

算法。我们发现，对于悲观算法（如 CQL [29]）和行为约束算法（如 IQL [28]），离线 Q 函数在在线微调初期会经历一个“重校准”阶段，其值发生显著变化。这一重校准阶段可能导致遗忘，甚至完全丧失离线初始化。当训练中没有离线数据时，这表现为价值函数的发散。即使是专门为微调设计的方法（如 CalQL [37]）在离线数据有限或缺失的情况下仍受此问题困扰。这扩展了 Nakamoto 等人 [37] 关于标准离线到在线微调设置中遗忘现象的观察，表明在无数据保留微调中，遗忘和丧失知识构成了更严峻的挑战。

能否从离线强化学习的价值函数与策略初始化出发进行微调，同时不保留离线数据且不遗忘预训练成果？本文的核心洞见在于：即使仅用少量适当采集的数据来“模拟”离线数据保留，且这些数据更贴近在线微调任务的分布，也能极大促进重校准，缓解预训练与微调之间的分布不匹配，并防止遗忘。一旦完成重校准，我们便可运行最高效的在线强化学习方法（无需悲观约束或行为约束），以实现样本高效的在线学习。本文提出的方法 **WSRL**（**Warm Start Reinforcement Learning**，热启动强化学习）通过引入预热阶段来实例化这一思想：先用预训练策略进行少量在线采样以初始化在线回放缓冲区，随后结合多种离线强化学习初始化运行最优在线强化学习方法进行微调。无论是否保留离线数据，**WSRL** 均能比现有算法学习更快，并达到更高的渐近性能。该方法并非特别新颖或巧妙的算法，但或许是实现离线初始化有效微调的最自然途径之一。

本文的主要贡献在于研究不保留离线数据的强化学习在线微调问题，我们将这一范式称为无保留微调。我们对现有离线到在线强化学习方法进行了详细分析，发现微调过程中通常需要离线数据来缓解 Q 值发散以及由分布偏移导致的遗忘，但保留离线数据也会降低渐近微调速度。我们证明了一种简单方法：引入预热阶段，用少量来自预训练离线强化学习策略的转移样本初始化回放缓冲区，然后运行简单的在线强化学习算法，即可实现样本高效的微调，且不遗忘预训练初始化。

2. 问题表述：无离线数据的微调

我们在无限时域马尔可夫决策过程（Markov Decision Process, MDP）中运行， $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathbf{P}, r, \gamma, \rho\}$ ，该过程由状态空间 $\mathcal{S}$ 、动作空间 $\mathcal{A}$ 、转移动态函数 $\mathbf{P}(s'|s, a) : \mathcal{S} \times \mathcal{A} \mapsto \mathcal{P}(\mathcal{A})$ 、奖励函数 $r : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ 、折扣因子 $\gamma \in [0, 1)$ 和初始状态分布 $\rho : \mathcal{P}(\mathcal{S})$ 组成。我们可以使用离线强化学习预训练策略 $\pi_{\psi}^{\text{pre}}(a|s) : \mathcal{S} \mapsto \mathcal{P}(\mathcal{A})$ 和预训练 Q 函数 $Q_{\theta}^{\text{pre}}(s, a) : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ 。我们的目标是构建一种在线微调算法，该算法仅使用 $\pi_{\psi}^{\text{pre}}(a|s)$ 和 $Q_{\theta}^{\text{pre}}(s, a)$ ，而不使用 $\mathcal{D}_{\text{off}}$ ，以最大化折扣回报： $\eta(\pi) = \mathbb{E}_{s_{t+1} \sim \mathbf{P}(\cdot|s_t, a_t), a_t \sim \pi(\cdot|s_t), s_0 \sim \rho} \sum_{t=0}^{\infty} [\gamma^t r(s_t, a_t)]$ 。

问题设置。 请注意，本文研究的强化学习微调问题不允许保留 $\mathcal{D}_{\text{off}}$ 。我们将此问题设置称为无保留在线微调。从概念上讲，我们的问题设置接近离线到在线微调问题 [36, 28, 37]，但我们不允许数据保留。

3. 理解离线数据在在线微调中的作用

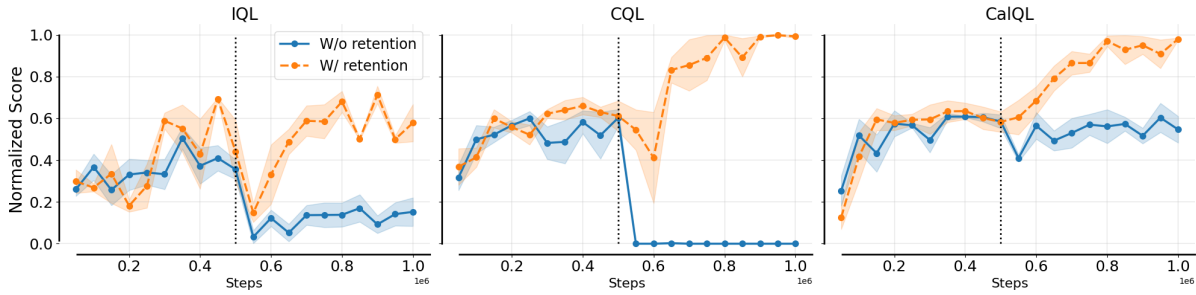


图 2：在无保留微调中，IQL、CQL 和 CalQL 均无法在 kitchen-partial 任务上进行微调。相比之下，在微调期间持续使用离线数据进行训练时，这些算法能够按预期工作。垂直虚线表示预训练和微调之间的分界。

我们首先尝试理解为什么当前的离线到在线强化学习方法需要保留离线数据集。特别是，我们希望理解保留离线数据的优缺点，并从中获得洞见，以开发能够发挥类似作用但无需保留离线数据的新方法。我们围绕两个轴进行研究：（1）分析在微调开始时保留离线数据的作用，（2）分析保留离线数据对渐近微调性能的影响。

3.1. 离线数据在微调开始时的作用

扩展先前工作的观察结果 [37]，我们发现如果不保留离线数据，离线强化学习初始化的微调会严重失败。具体来说，在图 2 中观察到，离线强化学习算法 IQL [28] 和 CQL [29] 在微调一开始就出现遗忘，性能下降到 D4RL [14] 中 kitchen-partial 任务上接近 0% 的成功率。更重要的是，它们在微调过程中无法恢复。CalQL [37] 是一种通过学习校准 Q 函数专门为后续微调设计的离线强化学习方法，其性能最初会下降，但随着进一步的在线训练而提高。尽管如此，它仍然难以超越其预训练性能。

为了更好地理解上述结果，我们引入一些术语。我们将“反学习”定义为微调开始时的性能下降，之后可能恢复；将“遗忘”定义为微调开始时预训练初始化的破坏，使得在线强化学习训练几乎无法恢复。一般而言，由于离线与在线之间状态-动作对的分布偏移，反学习可能不可避免（例如，考虑一个稀疏奖励问题，策略动作的微小变化导致回报的巨大变化）[58]。另一方面，完全遗忘预训练初始化是有问题的，因为这抵消了离线强化学习预训练的好处。我们的目标是让智能体在反学习后快速恢复，这依赖于对预训练知识的有效利用。观察图 2，虽然所有算法都发生反学习，但 CQL 和 IQL 发生遗忘。而 CalQL 没有遗忘，它

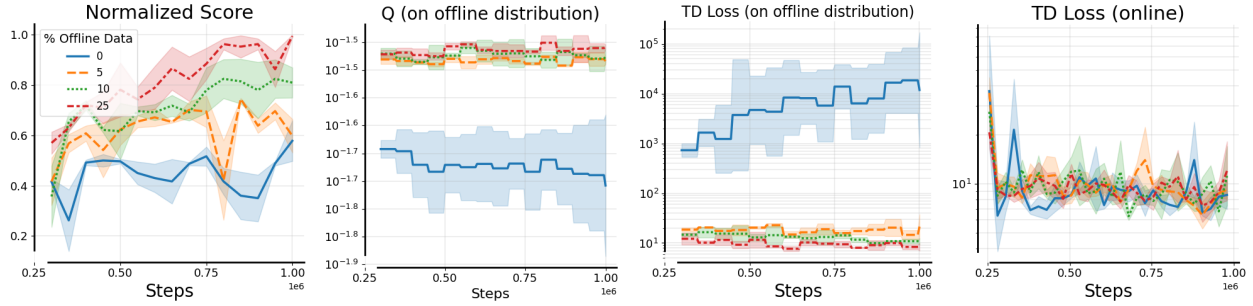


图 3: 当微调期间移除离线数据（不同程度）时，性能下降（子图 a），因为拟合在离线数据集分布上的 Q 函数发散（子图 b、c），即使 Q 函数能够拟合在线分布（子图 d）。该图展示了在 kitchen-partial 上微调 CalQL，每个更新批次中保留 0/5/10/25% 的离线数据。我们在 IQL 和 CQL 上也有类似发现。

没有进一步改进。这表明在没有离线数据的情况下，使用在线强化学习进行微调存在瓶颈，不同的离线强化学习初始化在不同程度上受到这一挑战的影响。

为什么不保留离线数据会造成损害？ 为了建立对可能出现问题的直觉，我们在图 3 中观察保留不同数量离线数据时发生的情况。图 3(b) 显示，随着保留的离线数据量减少，离线分布下的平均 Q 值开始发散。这种 Q 值发散反过来对应于 TD 误差的发散（图 3(c)），可能突显了遗忘。我们将在第 5 节展示如何通过我们的方法防止这种发散。

深入探究后，我们发现这种发散仅出现在离线数据分布下（我们在此分布上评估度量但不进行训练）：在线分布上的时序差分误差无论保留多少离线数据都保持较小（图 3(d)）；另一方面，离线数据分布下的时序差分误差随着离线数据的减少而显著增大（图 3(c)）。这一趋势在 CQL、IQL 和 CalQL 中一致，尽管 CalQL 的发散程度最轻，这或许得益于其校准的 Q 函数，这与 CalQL 在此问题设置中的稳定性和最佳性能相关，如图 2 所示。

图 2。 这表明当前离线到在线微调算法中不保留数据的问题可能源于在线 rollout 数据与离线数据分布之间的一种分布偏移。在更多同策略数据上微调会破坏我们对离线数据的拟合程度。如图 2 所示，这可能导致遗忘预训练初始化。

Takeaway 1: Distribution shift between offline and online data destroys Q-function fit

在微调期间仅使用在线经验而不保留离线数据，会破坏模型对离线数据的拟合程度：尽管在线数据上的时序差分误差与保留离线数据时相当，但离线分布下的时序差分误差会变得更大。

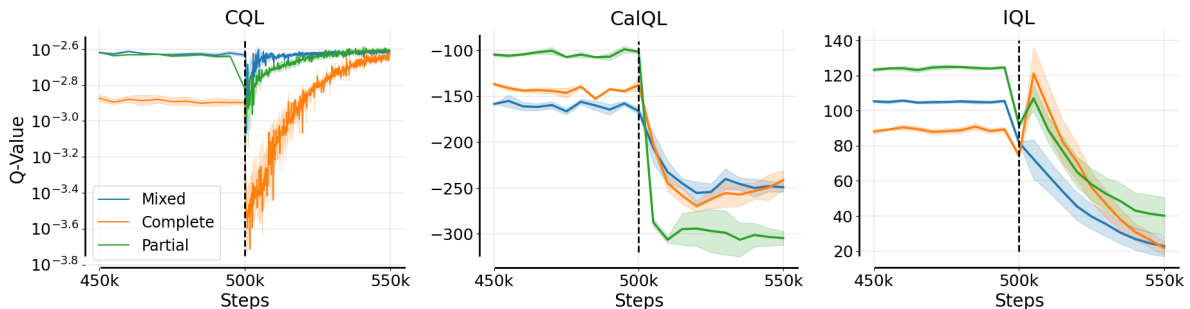


图 4: 在 kitchen-mixed、kitchen-complete 和 kitchen-partial 上无保留微调中，CQL（左）、CalQL（中）和 IQL（右）Q 函数的向下螺旋效应：当微调在 50 万步开始时，Q 函数进入向下螺旋。当它最终恢复时，策略已经遗忘（图 2）。

为什么 Q 值被低估？ 不仅离线数据分布下的 Q 值发散，在线分布上的 Q 值在微调开始时也会经历低估（见图 4）。这种 Q 值发散是离线强化学习与在线微调边界处“重新校准”过程的表现，此前在 Nakamoto 等人 [37] 的图 3 中已识别。然而，与 Nakamoto 等人 [37] 不同，无保留微调中的重新校准过程必须完全在有限的同策略数据上进行。因此，我们看到尽管 CalQL 显式修改了离线 Q 函数初始化的尺度，但在不保留离线数据时仍无法防止遗忘。

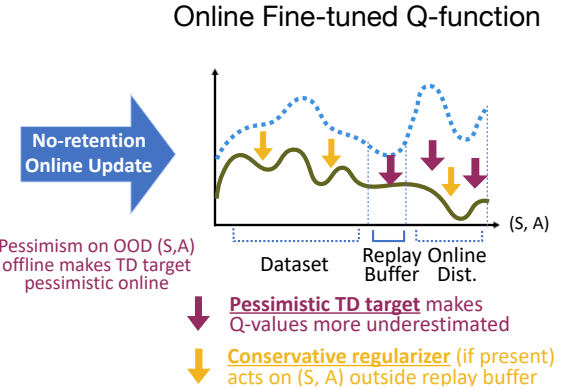
接下来，我们希望理解为何重新校准会导致 Q 值发散。图 5 为这种 Q 值低估现象提供了直观解释。考虑从环境中收集的第一批在线轨迹。

在线状态-动作对上估计时序差分误差的目标值计算会查询离线 Q 函数， Q_{θ}^{pre} ，在分布外状态-动作对上。

由于保守的离线强化学习预训练（例如 CQL 或 CalQL），这些分布外状态-动作对上学习到的 Q 值预计较小。在贝尔曼备份中使用这种悲观值作为时序差分目标，反过来会将这些低估误差传播到新的在线状态-动作对上，图 3(d) 证实了这些时序差分目标的有效传播。此外，

如果在微调期间继续运行悲观强化学习算法（例如 CQL 或 CalQL），保守正则化器会继续最小化分布外 Q 值：由于在线策略样本很少，未见动作的 Q 值会持续变小。

图 5：说明无保留微调中 Q 值为何被低估，并可能导致“螺旋式下降”的示意图。



这种机制性理解暗示了微调开始时学习到的 Q 函数中存在一种“螺旋式下降”。到 Q 函数恢复时，策略已经忘记了其预训练知识，无法再恢复到离线性能（图 2）。我们发现，随着微调中离线数据量的减少，这种现象变得更加有害，如图 3 所示。当无法保留任何离线数据时，最不利的影响就会出现。即使是经过校准的算法，如 CalQL，虽然更稳健，但仍然受到此问题的影响（图 4 中间）。

Takeaway 2: Re-calibration of Q-values leads to excessive underestimation

我们发现，微调开始时的 Q 值重新校准由于使用过度悲观的时序差分目标进行备份而导致过度低估。我们称之为“螺旋式下降”。

3.2. 离线数据对渐近性能的不利影响

如上所述，离线数据在当前离线到在线算法中通过帮助重新校准和防止 Q 值发散发挥着重要作用。但一旦重新校准结束，它对长期性能有何影响？我们发现，继续在离线数据上训练会损害最终性能和效率。具体而言，我们在图 6 中发现，离线强化学习微调往往比从头开始的在线强化学习算法（用离线数据初始化回放缓冲区 [3, 51]）慢得多。这相当令人担忧，因为它表明要么离线强化学习预训练对微调没有好处（不像机器学习的其他领域，预训练有显著帮助），要么现有的强化学习微调方法

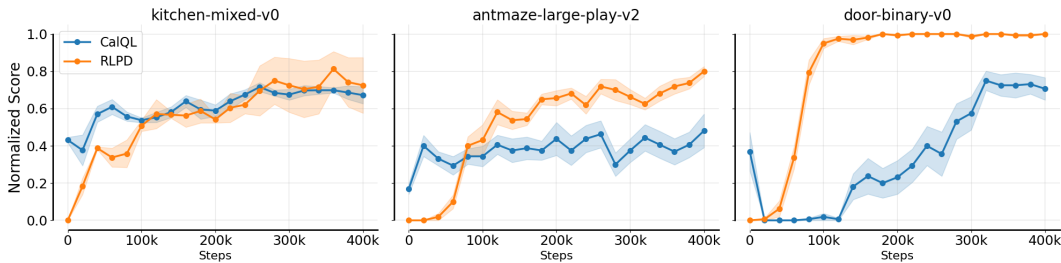


图 6：保留离线数据并非高效之举，且在三个不同环境中被 RLDP 等在线强化学习方法超越。RLDP 从零开始，而 CalQL 从步骤 0 的预训练初始化开始。

来自各种离线强化学习初始化的效果不足以充分利用离线初始化。本文将展示，在高更新数据比（UTD）机制下对在线强化学习方法进行简单修改，即可有效利用多种离线强化学习方法提供的初始化，而无需离线数据。

Takeaway 3: Retaining offline data hurts asymptotic performance

虽然保留离线数据似乎对于当前微调方法在微调初期防止遗忘至关重要，但继续使用（悲观的）离线强化学习算法对这些离线数据进行更新会对渐近性能和效率产生负面影响。

4. WSRL：无需离线数据保留的快速微调

到目前为止，我们看到在离线强化学习算法中保留离线数据会减慢在线微调的速度，但由于遗忘问题，我们也不能移除离线数据。我们如何既能解决离线初始化的遗忘问题，又能在线达到渐近样本效率？

核心思想。或许解决渐近效率问题的一个直接方法是采用标准的在线强化学习方法，在微调阶段不使用悲观约束或额外限制，这与当前离线到在线微调方法在微调阶段仍保留离线强化学习特定技术不同。我们可以通过在高更新数据比（UTD）机制下运行来进一步加速在线学习 [3, 6]。剩下的问题是：在没有离线数据的情况下，如何应对微调初期出现的灾难性遗忘，以免阻碍在线阶段的进一步提升？我们的洞见是，可以在在线微调初期，使用冻结的离线强化学习策略收集少量预热转移样本，从而“模拟”对离线数据的持续训练。在不保留离线数据的情况下，通过激进的高更新数据比在线强化学习方法对这些转移样本进行训练，可以缓解灾难性遗忘的挑战。我们的方法 **WSRL (Warm Start Reinforcement Learning, 热启动强化学习)** 将这些洞见实例化为一种极其简单且实用的方法，使我们无需离线数据即可获得强大的微调结果。

WSRL 算法。WSRL 是一种异策略演员-评论家算法（算法 1）。它使用预训练的 Q 函数 Q_{θ}^{pre} 初始化价值函数和策略，这些预训练模型可以来自任何离线强化学习算法 π_{ψ}^{pre} 。然后，WSRL 利用前 K 个在线步骤，使用冻结的离线强化学习策略收集少量轨迹，以模拟对离线数据的保留。我们将此阶段称为“预热”阶段。在预热数据收集之后，WSRL 使用标准时序差分（TD）更新和策略梯度来训练价值函数和策略。在微调阶段，我们采用高更新数据比（UTD）[13, 6] 进行微调，并遵循其他最佳实践。为了应对高 UTD 机制下的过估计等问题 [20]，我们在演员和评论家中都使用了 Q 函数集成 [6] 和层归一化 [21]。

实现细节。本文中的大多数结果使用 CalQL 来初始化 WSRL，尽管原则上也可以使用其他初始化方式。关于使用不同初始化运行 WSRL 的内容，请参见附录 H。我们选择软演员-评论家（Soft Actor-Critic）[18] 作为在线微调算法，并使用 10 个 Q 网络的集成和层归一化。我们使用的 UTD 为 4。这一设计受到 Ball 等人 [3] 工作的启发。我们使用

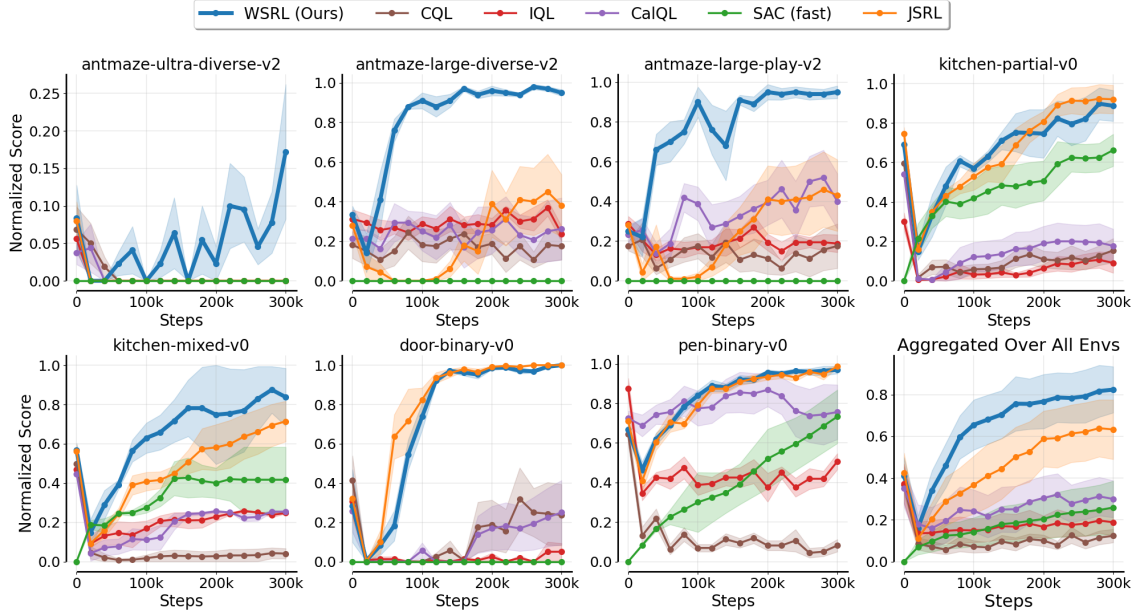


图 7: 在无保留微调中, WSRL 高效微调并大幅超越所有先前算法, 这些算法往往无法从初始性能下降中恢复。最接近的基线 JSRL 使用了类似于预热的数据收集技术。

$K = 5000$ 微调开始时的预热步骤。更多实现细节见附录 I。

5. 实验评估

本实验旨在研究 WSRL 在不保留离线数据的情况下进行在线微调的效果。我们还对 WSRL 中的设计决策进行消融, 以理解其有效性的原因。具体而言, 我们研究以下问题: (1) WSRL 能否在无保留设置下实现高效微调?

(2) WSRL 与保留离线数据的方法相比如何? (3) WSRL 中预热阶段有多关键? (4) 使用在线强化学习算法进行在线微调有多重要? (5) 预训练策略、价值函数或两者都预训练有多重要?

5.1. 基线与实验设置

虽然大多数先前的强化学习微调方法并非专为无保留微调设置而设计, 但它们确实可以应用于或改造用于我们的设置。JSRL [55] 在每个回合中使用预训练策略先执行若干步, 然后用当前策略继续执行。在线策略使用回滚和继续执行的经验从头开始训练。为了提高 JSRL 的竞争力, 我们还用预训练策略初始化在线策略, 并以高更新数据比运行。离线强化学习方法也被证明可以进行在线微调。我们考虑三种离线方法: CQL [29]、IQL [28] 和 CalQL [37], 其中 CalQL 是 CQL 的扩展, 通过校准 Q 值以实现高效微调。另一种方法 **SO2** [61] 试图在微调期间通过高更新数据比和扰动值更新来平衡奖励最大化与悲观预训练。最后, RLPD [3] 是一种高效的在线强化学习算法, 它通过在每个更新批次中对离线数据集和在线缓冲区进行 50/50 采样来从头学习。虽然典型的微调方案涉及从离线数据集和在线回放缓冲区中采样每个更新批次 (CQL、CalQL、RLPD), 或用离线数据集初始化回放缓冲区 (IQL、**SO2**), 但我们在无保留设置下仅从在线缓冲区采样来评估它们。由于没有 50/50 采样的 RLPD 等同于快速更新的软演员-评论家 [18] 智能体, 我们将其称为 SAC (快速版)。

实验设置。我们按照先前工作 [37, 28, 29] 使用的协议, 研究了一系列具有挑战性的基准任务和预训练数据集组合。我们在 D4RL [12] 的 Antmaze、Kitchen 和 Adroit 任务以及 Gym MuJoCo 运动任务 ¹ 上进行实验。更多讨论见附录 C。

¹ 结果见附录 B。

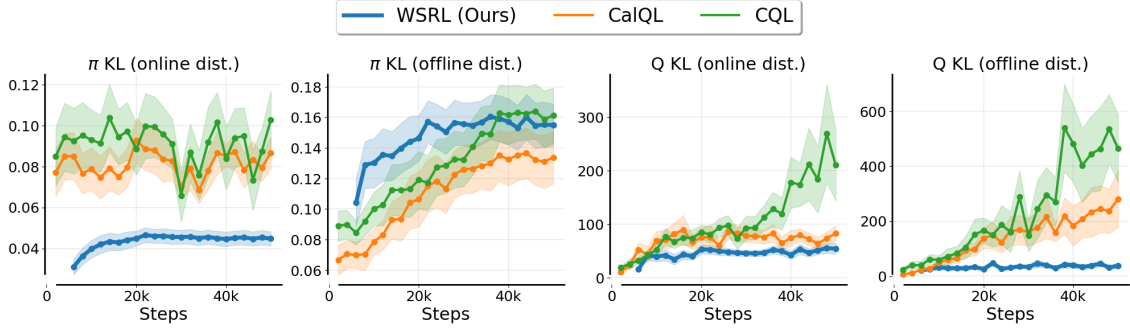


图 8: 预训练策略和 Q 函数与微调后策略和 Q 函数之间的 KL 散度表明, WSRL 没有忘记其预训练。四个图分别显示了在线和离线状态分布上的 $D_{KL}(\pi_{\psi}^{\text{pre}} || \pi_{\psi})$ 和 $D_{KL}(\text{softmax}(Q_{\theta}^{\text{pre}}) || \text{softmax}(\pi_{\psi}))$ 。该图汇总了六个不同环境的 KL 散度, 分别在图 21 和图 22 中展示, 并在附录 E 中有更详细的讨论。

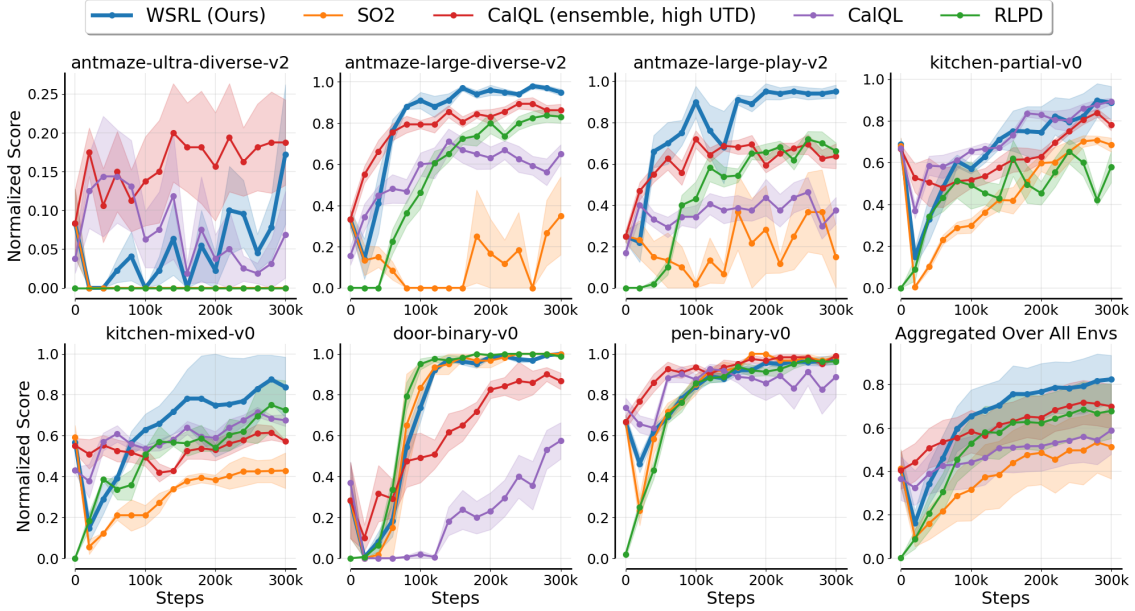


图 9: 与 *do* 在线保留离线数据的方法相比, WSRL 尽管没有保留任何离线数据, 却仍能更快或具有竞争力地进行微调, 这或许令人惊讶。这尤其凸显了预热的好处。

5.2. WSRL 能否在无保留微调中实现高效微调?

图 7 将 WSRL 与前述应用于无保留微调的方法进行了比较。在七项不同任务中, WSRL 显著优于基线方法, 微调速度更快, 且达到更高的渐近性能。CQL、IQL 和 CalQL 在此设置下完全失败, 如第 3 节所述。SAC (快速) 在探索密集型环境中完全失败, 但在某些环境中可以缓慢改进。最具竞争力的基线是 JSRL。虽然 JSRL 在 *Adroit* 上达到了与 WSRL 相同的性能, 但在 *Antmaze* 和 *Kitchen* 上明显较差。我们假设这种性能差距源于 WSRL 能够从预训练的价值初始化中受益, 尤其是在预训练 Q 函数良好的数据集上 (例如 *Antmaze*)。请注意, 虽然 WSRL 确实在性能上经历了初始的遗忘 (见图 20), 但它恢复迅速, 并优于其他通常根本无法恢复的方法。如前所述, 在某些环境中一定程度的遗忘可能是不可避免的, 但重要的是防止遗忘预训练的初始化。为了评估策略和 Q 函数的遗忘程度, 我们在图 8 中测量了遗忘期间预训练与微调 Q 函数之间以及预训练与微调策略之间的 KL 散度。虽然 CQL 和 CalQL 在 Q 函数上出现发散, 但 WSRL 的 Q 函数保持稳定, 并相对接近其预训练初始化。策略的 KL 散度表明 π_{ψ} 没有忘记其

在微调中相关状态上的预训练，同时遗忘可能无关的信息。这表明虽然 WSRL 最初会遗忘，但似乎并未真正忘记。更多细节见附录 E。

5.3. WSRL 与保留离线数据的方法相比如何？ 在图 9 中，我们将 WSRL 与在线微调期间仍保留并利用离线数据的先前方法进行了比较。例如，图中标记为 CalQL 的方法会从离线数据和在线数据中采样转移来构建更新批次。为了公平比较，我们还比较了所有方法在 UTD 为 4 和 10 个 Q 函数集成下训练的版本。观察到 WSRL 也优于这些方法，尽管这些先前方法在微调期间保留了整个离线数据集。具体而言，WSRL 通常比 CalQL 获得更高的渐近性能，并且微调速度更快，这表明保留离线数据可能不是渐近性能的最佳折衷方案，正如我们在第 3 节中所展示的。WSRL 还优于 RLPD，表明 WSRL 能够有效利用预训练的价值函数和策略初始化进行快速在线学习。

5.4. 预热阶段有多关键？ 我们发现预热阶段对于使用在线强化学习进行微调至关重要。如图 10 所示，没有预热的 WSRL 在三个不同环境中表现显著更差。此外，我们发现使用这种简单的预热方案优于使用相同数量的离线数据集转移进行初始化或完全保留离线数据集的 WSRL（见附录 K 和 L）。我们假设预热有助于缓解分布偏移，并防止微调开始时的发散和“螺旋式下降”。如图 11 所示，当采用预热时，学习到的 Q 值不会发散到过度悲观的值，并且 TD 损失在离线数据上也保持较小，避免了第 3 节中的“螺旋式下降”。更详细的讨论见附录 G。

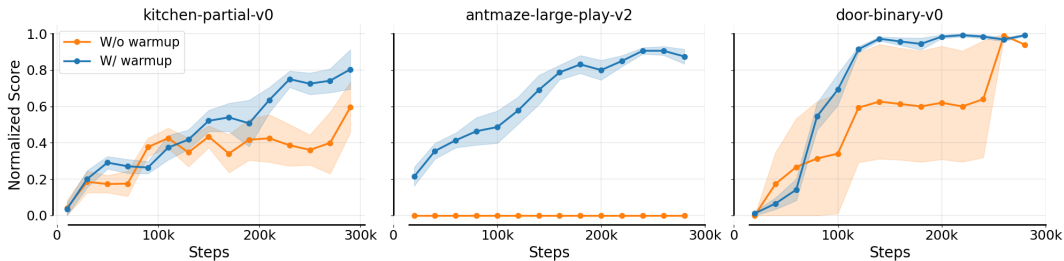


图 10：预热对于快速微调至关重要：当 WSRL 不使用初始 5000 步预热时，其性能更差或方差更高。

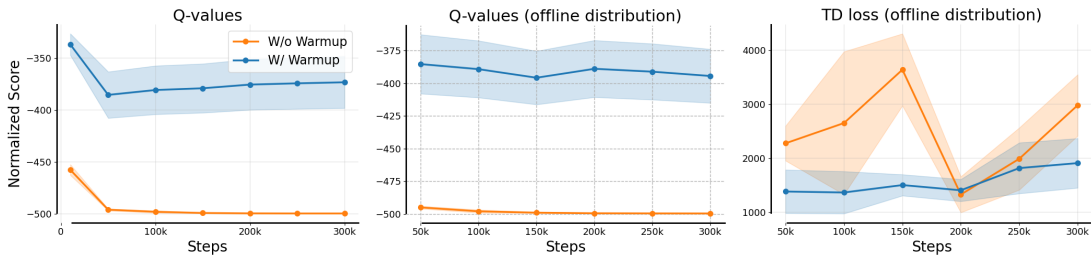


图 11：预热阶段有助于防止螺旋式下降：（左）微调期间的 Q 值，预热缓解了过度悲观的值；（中、右）在离线分布上评估的 Q 值和 TD 误差，预热防止了发散。数据来自 Antmaze-large-play 上的 WSRL。

5.5. 使用标准（非悲观）在线强化学习算法进行微调有多重要？

除了预热阶段，在 WSRL 中使用标准的在线强化学习算法进行微调也是一个关键的设计选择。我们通过尝试在微调期间使用离线强化学习算法来消融这一决策。这里我们选择使用 CalQL，因为与 CQL 和 IQL 相比，它在设计上不太可能出现 Q 值发散（第 3 节），但仍然是一种悲观算法。我们还使用预训练的

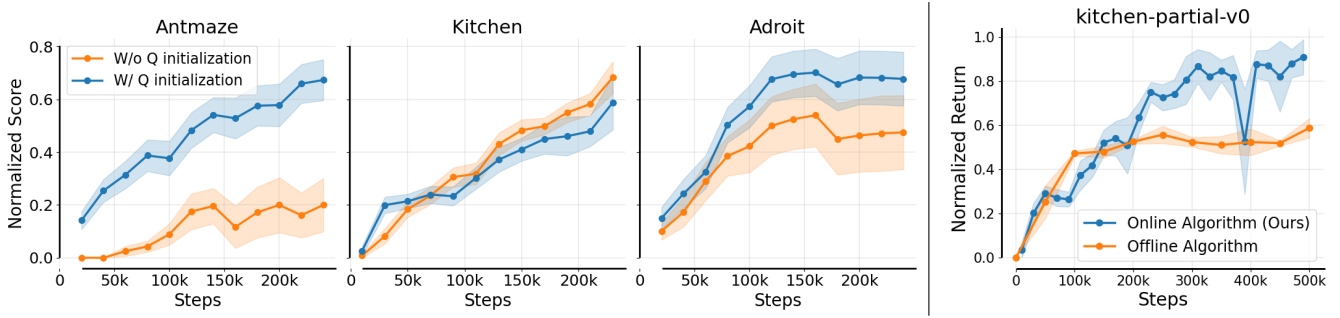


图 12：消融研究：使用 Q 函数初始化（左）和标准在线强化学习算法（右）而不带悲观或约束的重要性。左：当预训练数据集具有高覆盖率时（例如 Antmazes），Q 函数初始化尤其有帮助。每个图在域上对不同数据集类型进行平均；右：使用标准在线强化学习算法进行微调的重要性：SAC 的学习速度快于 CalQL。

策略和 Q 函数，并使用相同数量的在线预热步骤。如图 12（右）所示，使用离线算法明显比使用标准在线强化学习算法 SAC 差。

5.6. 初始化策略、价值函数或两者有多重要？

策略初始化的重要性。在在线微调开始时，WSRL 将策略初始化为预训练策略。由于预训练策略已经能够在环境中有意义地行动，它加快了在线学习。在图 13 中，我们比较了 WSRL 在有和没有“策略初始化”情况下的性能，发现使用预训练策略进行初始化对于快速微调至关重要。

Q 值初始化的好处。在

图 12（左）中，我们观察到，虽然在某些领域中初始化价值函数并未带来额外收益，但在其他领域中它加快了微调速度。例如，在蚁穴（Antmaze）领域中，使用 Q 函数进行初始化尤为有益。我们推测，这是因为蚁穴领域的预训练数据集相比灵巧手（Adroit）和厨房（Kitchen）领域具有更广泛的覆盖范围，从而产生了更优的离线 Q 函数。因此，在蚁穴领域中使用信息量更丰富的 Q 函数进行初始化，加速了在线微调。

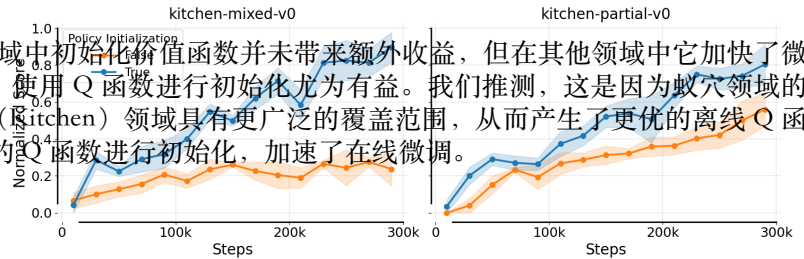


图 13：WSRL 中策略初始化的重要性：有了策略初始化，WSRL 在 Kitchen 中的表现要好得多。

5.7. WSRL 在真实世界机器人任务中的表现如何？

Task	WSRL @ 18 min	SERL @ 50 min	Offline Pretrained Cal-QL
Franka Peg Insertion	20/20	0/20	13/20

表 1：WSRL 实现了高效的现实世界强化学习微调。WSRL 在离线强化学习预训练的基础上迅速提升，并且微调速度远快于 RLPD/SERL。WSRL 仅需 11 分钟收集 5000 步预热数据，随后仅进行 7 分钟的在线强化学习即可掌握任务。使用 20 种不同的初始位姿进行评估。

真实世界机器人与任务设置。我们使用一台 7 自由度 Franka Emika Panda 机械臂，以 10 赫兹的频率进行末端执行器控制。如图 14 所示，我们使用两个腕部摄像头图像和机器人本体感受状态作为输入。任务“Franka 插销”要求旋转、导航并将带有销钉的塑料块插入盒状框架中对应的孔中，使用的 3D 打印物体来自 Luo 等人 [33]。

数据收集与环境。由于插销任务没有现成的预训练数据集，我们收集了一个约 17K 转移的小型数据集。该数据集通过保存在在线强化学习运行期间的回放缓冲区获得。因此，我们的数据集包含混合质量的数据，通常构成专家回放数据集 [12]。我们选择收集此类混合质量数据而非遥操作人类演示数据，以测试离线强化学习算法从次优数据中学习的能力。更多细节见附录 J。

真实世界结果。本文将 WSRL 与 SERL [31] 进行比较，后者是一种强大的先前方法，它在真实机器人上将离线数据整合到高效的在线强化学习训练中，并建立在 RLPD [4] 算法之上。我们对两种方法都进行了 20K 环境步的在线训练。WSRL 使用由 CalQL 在我们的离线数据集上训练的离线检查点进行初始化。为了进行公平比较，我们用相同的离线数据集初始化 SERL 的重放缓冲区。我们从均匀分布在目标姿态周围的初始状态开始进行 20 次试验评估，结果见表 1。

本文发现，在仅 18 分钟的实时强化学习训练中（大约 5K 预热步 + 3K 演员步，不包括预热为 7 分钟），WSRL 能够从 13/20 的离线性能微调到完美的 20/20。相比之下，本文发现 SERL/RLPD 在 50 分钟（20k 演员步）结束时达到 0/20。从定性角度看，本文观察到 SERL 经常卡在分布外位置，只是停止而无法进一步推进（示例见图 15）。即使在离线分布的支持范围内运行，SERL 也经常在各种位置上方悬停而不采取插入路径，这表明它无法从离线数据集中学习。相比之下，WSRL 能够有效利用其离线强化学习预训练，并从该知识中引导以快速在线微调。

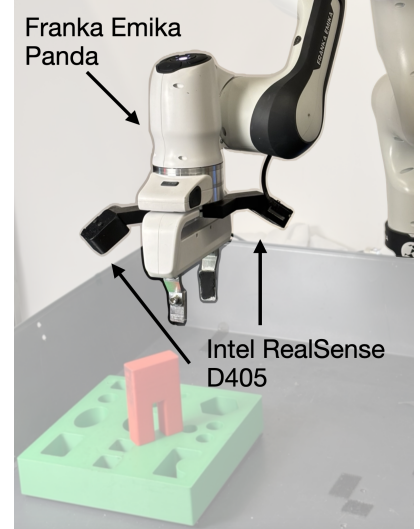
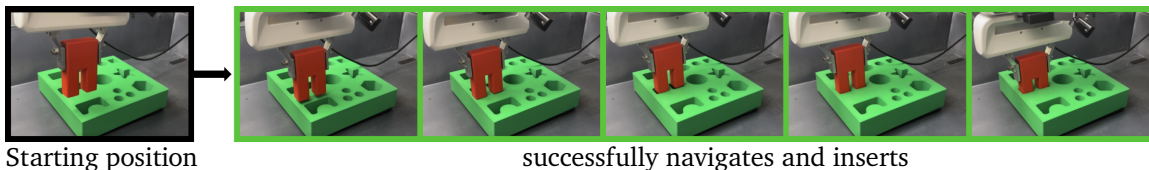


图 14：用于插销实验的 Franka 机器人设置。

WSRL Successful Insertion



RLPD Failure Mode

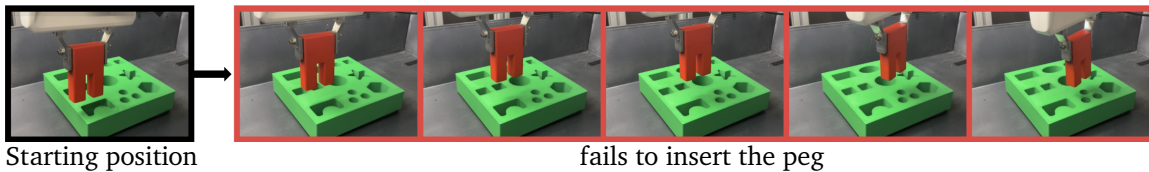


图 15：WSRL（绿色）学习到专家级策略，可以从任何位置成功插入。SERL（红色）未能从离线数据集中学习到成功策略，并漂移到分布外位置，然后停滞不前。

6. 相关工作

离线到在线强化学习。离线到在线强化学习专注于利用离线数据集尽可能高效地进行在线强化学习 [30, 36]。为此设置开发的许多方法利用离线预训练，然后在离线与在线数据的混合上进行专门的微调阶段 [36, 28, 1, 22, 42, 37]。离线强化学习方法也可以直接用于微调，方法是在添加新的在线数据时继续训练

至离线数据缓冲区 [29, 28, 53]。与本文目标最为相似的是 Agarwal 等人 [1] 的工作，其尝试利用先前的强化学习计算结果作为下游任务更优的初始化。然而，该工作以及上述所有方法仍要求在数据缓冲区中保留全部预训练数据。正如本文所示，这些方法在缓冲区中缺少离线数据时完全失效。本文方法不保留离线数据。Uchendu 等人 [55] 利用预训练策略引导在线微调，无需保留离线数据，但丢弃了价值函数，而价值函数在大多数演员-评论家强化学习算法中通常驱动学习过程。Ji 等人 [24] 与 Luo 等人 [35] 遵循串联学习 [39] 的思想，在共享重放缓冲区上同时运行离线强化学习与在线强化学习。尽管该系列工作的高层动机原则同样在于利用离线强化学习提升在线强化学习效率，但其中并不涉及预训练。

离线强化学习策略在线微调中的瓶颈。本文表明，保留离线数据能极大稳定微调初期 Q 函数的重新校准过程，否则状态-动作分布偏移可能导致灾难性遗忘。Luo 等人 [34] 观察到将离线数据放入离线强化学习重放缓冲区可稳定微调，但同时会减慢学习速度。然而，离线数据为何损害微调仍不清楚，本文分析旨在回答该问题。Lee 等人 [30] 识别出离线数据与在线采样数据之间存在状态-动作分布偏移，但未明确分析该偏移在在线微调中的负面影响。Nakamoto 等人 [37] 表明离线预训练 Q 函数的尺度校准不佳是悲观算法在保留离线数据的在线微调中不稳定的关键原因。本文分析将该分析扩展至不保留离线数据的情形，并揭示了一个不同的原因（状态-动作分布偏移），该原因同样困扰着 Nakamoto 等人 [37] 的方法在此情形下的表现。

带有先验数据但无预训练的在线强化学习。另一条研究路线完全绕过了离线强化学习预训练，直接使用纯在线强化学习智能体，从零开始同时利用离线数据和在线交互数据样本进行学习 [50, 62, 3]。尽管未采用预训练，有证据表明这种方法可以表现得相当好，通常优于使用独立离线预训练阶段的在线强化学习微调方法。如果利用先前数据最有效的方式是将其直接放入回放缓冲区而不进行任何预训练——无论使用哪种预训练算法——那么这或许表明，我们缺少某些关键要素，无法构建真正可扩展的强化学习预训练与微调范式。本文表明，问题至少很大一部分在于离线强化学习初始化的在线微调，并构建了一种极其简单的方法来解决该问题。

无数据保留的强化学习策略微调。最后，由于环境动态和任务规范中的非平稳性假设，许多持续学习和终身强化学习方法也在不保留先前经验的情况下微调策略 [46, 26, 23, 56, 40]。元强化学习方法 [11, 47, 52, 44, 2, 9, 16] 假设可以访问任务/环境分布，以优化在线快速微调。相比之下，本文仅考虑单环境、单任务设定，其中预训练和微调在同一环境中针对同一任务进行。在相同的单环境、单任务设定下，许多先前工作研究在线策略强化学习方法（例如近端策略优化（PPO）[49]）来微调预训练策略 [48, 27, 43, 17, 57, 45]。其中，Wolczyk 等人 [57] 也观察到微调初期存在遗忘现象，并发现利用持续学习技术显式缓解遗忘可以提高微调效率。与这些工作不同，本文关注异策略演员-评论家强化学习方法，这类方法具有更高的样本效率，但需要不同的解决策略来应对这一遗忘问题。

7. 结论

本文探讨了在不保留任何离线数据集且不与其共同训练的情况下，在线微调强化学习智能体的可能性。这种设定对于真正可扩展的强化学习至关重要：先使用离线强化学习在多样化数据集上进行预训练，随后进行在线强化学习微调，而保留离线数据代价高昂或不可行。我们发现，以往的离线到在线强化学习算法在此设定下完全失效，原因在于

分布偏移导致的 Q 值发散。然而，如果仅使用在线强化学习算法进行微调，并通过预热阶段使 Q 值稳定，则可以防止 Q 值发散。希望 WSRL 能够揭示无保留微调中的挑战，并启发未来对无保留强化学习微调这一重要范式的研究。

致谢

感谢 Seohong Park、Mitsuhiko Nakamoto、Kyle Stachowicz 以及匿名审稿人对本文早期版本提供的有益讨论和反馈。本研究得到了 AI 研究所和海军研究办公室的资助，项目编号为 N00014-24-1-2206 和 N00014-20-1-2383。感谢 TPU 研究云（TRC）和谷歌云提供的慷慨计算资源捐赠，使这项工作得以完成。

参考文献

- [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Reincarnating reinforcement learning: Reusing prior computation to accelerate progress. *Advances in neural information processing systems*, 35:28955–28971, 2022.
- [2] Karol Arndt, Murtaza Hazara, Ali Ghadirzadeh, and Ville Kyrki. Meta reinforcement learning for sim-to-real domain adaptation. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 2725–2731. IEEE, 2020.
- [3] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. *arXiv preprint arXiv:2302.02948*, 2023.
- [4] Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. *arXiv preprint arXiv:2302.02948*, 2023.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=AY8zfZm0tDd>.
- [7] C. Cheng, T. Xie, N. Jiang, and A. Agarwal. Adversarially Trained Actor Critic for Offline RL. *ICML*, 2022.
- [8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [9] Ron Dorfman, Idan Shenfeld, and Aviv Tamar. Offline meta reinforcement learning—identifiability challenges and effective data collection strategies. *Advances in Neural Information Processing Systems*, 34:4607–4618, 2021.
- [10] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [11] Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2 : Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.

- [12] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine. D4RL: Datasets for deep data-driven reinforcement learning. In <https://github.com/rail-berkeley/d4rl>, 2020. URL <https://arxiv.org/pdf/2004.07219>.
- [13] Justin Fu, Aviral Kumar, Matthew Soh, and Sergey Levine. Diagnosing bottlenecks in deep q-learning algorithms. In *Proceedings of the 36th International Conference on Machine Learning*. PMLR, 2019.
- [14] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- [15] Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. *arXiv preprint arXiv:2106.06860*, 2021.
- [16] Jake Grigsby, Linxi Fan, and Yuke Zhu. Amago: Scalable in-context reinforcement learning for adaptive agents. *arXiv preprint arXiv:2310.09971*, 2023.
- [17] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. *arXiv preprint arXiv:1910.11956*, 2019.
- [18] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *arXiv*, 2018. URL <https://arxiv.org/pdf/1801.01290.pdf>.
- [19] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.
- [20] Hado van Hasselt. Double q-learning. In *Proceedings of the 23rd International Conference on Neural Information Processing Systems - Volume 2*, 2010.
- [21] Takuya Hiraoka, Takahisa Imagawa, Taisei Hashimoto, Takashi Onishi, and Yoshimasa Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=xCVJMsPv3RT>.
- [22] Hengyuan Hu, Suvir Mirchandani, and Dorsa Sadigh. Imitation bootstrapped reinforcement learning. *arXiv preprint arXiv:2311.02198*, 2023.
- [23] Yizhou Huang, Kevin Xie, Homanga Bharadhwaj, and Florian Shkurti. Continual model-based reinforcement learning with hypernetworks. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 799–805. IEEE, 2021.
- [24] Tianying Ji, Yu Luo, Fuchun Sun, Xianyuan Zhan, Jianwei Zhang, and Huazhe Xu. Seizing serendipity: Exploiting the value of past success in off-policy actor-critic. *arXiv preprint arXiv:2306.02865*, 2023.
- [25] Zhengyao Jiang, Tianjun Zhang, Michael Janner, Yueying Li, Tim Rocktäschel, Edward Grefenstette, and Yuandong Tian. Efficient planning in a compact latent action space. *arXiv preprint arXiv:2208.10291*, 2022.
- [26] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13): 3521–3526, 2017.

- [27] Jens Kober and Jan Peters. Policy search for motor primitives in robotics. *Advances in neural information processing systems*, 21, 2008.
- [28] Ilya Kostrikov, Jonathan Tompson, Rob Fergus, and Ofir Nachum. Offline reinforcement learning with fisher divergence critic regularization. *arXiv preprint arXiv:2103.08050*, 2021.
- [29] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33:1179–1191, 2020.
- [30] Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. Offline-to-online reinforcement learning via balanced replay and pessimistic Q-ensemble. In *Conference on Robot Learning*, pages 1702–1712. PMLR, 2022.
- [31] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. *arXiv preprint arXiv:2401.16013*, 2024.
- [32] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *arXiv preprint arXiv:2401.16013*, 2024.
- [33] Jianlan Luo, Charles Xu, Fangchen Liu, Liam Tan, Zipeng Lin, Jeffrey Wu, Pieter Abbeel, and Sergey Levine. Fmb: a functional manipulation benchmark for generalizable robotic learning. *The International Journal of Robotics Research*, 44(4):592–606, 2025.
- [34] Yicheng Luo, Jackie Kay, Edward Grefenstette, and Marc Peter Deisenroth. Finetuning from offline reinforcement learning: Challenges, trade-offs and practical solutions. *arXiv preprint arXiv:2303.17396*, 2023.
- [35] Yu Luo, Tianying Ji, Fuchun Sun, Jianwei Zhang, Huazhe Xu, and Xianyuan Zhan. Offline-boosted actor-critic: Adaptively blending optimal historical behaviors in deep off-policy rl. *arXiv preprint arXiv:2405.18520*, 2024.
- [36] Ashvin Nair, Abhishek Gupta, Murtaza Dalal, and Sergey Levine. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- [37] Mitsuhiro Nakamoto, Simon Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [38] Evgenii Nikishin, Max Schwarzer, Pierluca D’Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *International conference on machine learning*, pages 16828–16847. PMLR, 2022.
- [39] Georg Ostrovski, Pablo Samuel Castro, and Will Dabney. The difficulty of passive learning in deep reinforcement learning. *Advances in Neural Information Processing Systems*, 34:23283–23295, 2021.
- [40] Sam Powers, Eliot Xing, Eric Kolve, Roozbeh Mottaghi, and Abhinav Gupta. CORA: Benchmarks, baselines, and metrics as a platform for continual reinforcement learning agents. In *Conference on Lifelong Learning Agents*, pages 705–743. PMLR, 2022.
- [41] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

- [42] Rafael Rafailov, Kyle Beltran Hatch, Victor Kolev, John D Martin, Mariano Phielipp, and Chelsea Finn. Moto: Offline pre-training to online fine-tuning for model-based robot learning. In *Conference on Robot Learning*, pages 3654–3671. PMLR, 2023.
- [43] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.
- [44] Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In *International conference on machine learning*, pages 5331–5340. PMLR, 2019.
- [45] Allen Z Ren, Justin Lidard, Lars L Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. *arXiv preprint arXiv:2409.00588*, 2024.
- [46] Mark Bishop Ring. *Continual learning in reinforcement environments*. The University of Texas at Austin, 1994.
- [47] Jonas Rothfuss, Dennis Lee, Ignasi Clavera, Tamim Asfour, and Pieter Abbeel. Promp: Proximal meta-policy search. *arXiv preprint arXiv:1810.06784*, 2018.
- [48] Stefan Schaal. Learning from demonstration. *Advances in neural information processing systems*, 9, 1996.
- [49] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [50] Yuda Song, Yifei Zhou, Ayush Sekhari, J Andrew Bagnell, Akshay Krishnamurthy, and Wen Sun. Hybrid RL: Using both offline and online data can make RL efficient. *arXiv preprint arXiv:2210.06718*, 2022.
- [51] Yuda Song, Yifei Zhou, Ayush Sekhari, Drew Bagnell, Akshay Krishnamurthy, and Wen Sun. Hybrid RL: Using both offline and online data can make RL efficient. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=yyBis80iUuU>.
- [52] Bradly C Stadie, Ge Yang, Rein Houthooft, Xi Chen, Yan Duan, Yuhuai Wu, Pieter Abbeel, and Ilya Sutskever. Some considerations on learning to explore via meta-reinforcement learning. *arXiv preprint arXiv:1803.01118*, 2018.
- [53] Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. Revisiting the minimalist approach to offline reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [54] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [55] Ikechukwu Uchendu, Ted Xiao, Yao Lu, Banghua Zhu, Mengyuan Yan, Joséphine Simon, Matthew Bennis, Chuyuan Fu, Cong Ma, Jiantao Jiao, et al. Jump-start reinforcement learning. In *International Conference on Machine Learning*, pages 34556–34583. PMLR, 2023.

- [56] Maciej Wołczyk, Michał Zając, Razvan Pascanu, Łukasz Kuciński, and Piotr Miłoś. Continual world: A robotic benchmark for continual reinforcement learning. *Advances in Neural Information Processing Systems*, 34:28496–28510, 2021.
- [57] Maciej Wołczyk, Bartłomiej Cupiał, Mateusz Ostaszewski, Michał Bortkiewicz, Michał Zając, Razvan Pascanu, Łukasz Kuciński, and Piotr Miłoś. Fine-tuning reinforcement learning models is secretly a forgetting mitigation problem. *arXiv preprint arXiv:2402.02868*, 2024.
- [58] Tengyang Xie, Nan Jiang, Huan Wang, Caiming Xiong, and Yu Bai. Policy finetuning: Bridging sample-efficient offline and online reinforcement learning. *Advances in neural information processing systems*, 34:27395–27407, 2021.
- [59] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11975–11986, 2023.
- [60] Haichao Zhang, We Xu, and Haonan Yu. Policy expansion for bridging offline-to-online reinforcement learning. *arXiv preprint arXiv:2302.00935*, 2023.
- [61] Yinmin Zhang, Jie Liu, Chuming Li, Yazhe Niu, Yaodong Yang, Yu Liu, and Wanli Ouyang. A perspective of q-value estimation on offline-to-online reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 16908–16916, 2024.
- [62] Yifei Zhou, Ayush Sekhari, Yuda Song, and Wen Sun. Offline data enhanced on-policy policy gradient with provable guarantees. *arXiv preprint arXiv:2311.08384*, 2023.
- [63] Zhiyuan Zhou, Pranav Atreya, Abraham Lee, Homer Walke, Oier Mees, and Sergey Levine. Autonomous improvement of instruction following skills via foundation models. *arXiv preprint arXiv:2407.20635*, 2024.

附录

A. Antmaze 环境上的额外结果

在正文中，我们展示了三个最具挑战性的 antmaze 环境上的结果。在此，除了图 7 和图 9 所示的三个 antmaze 环境外，我们还提供了 WSRL 在所有八个 D4RL antmaze 环境上的结果，并与强基线方法进行了比较。结果表明，WSRL 显著优于基线方法。

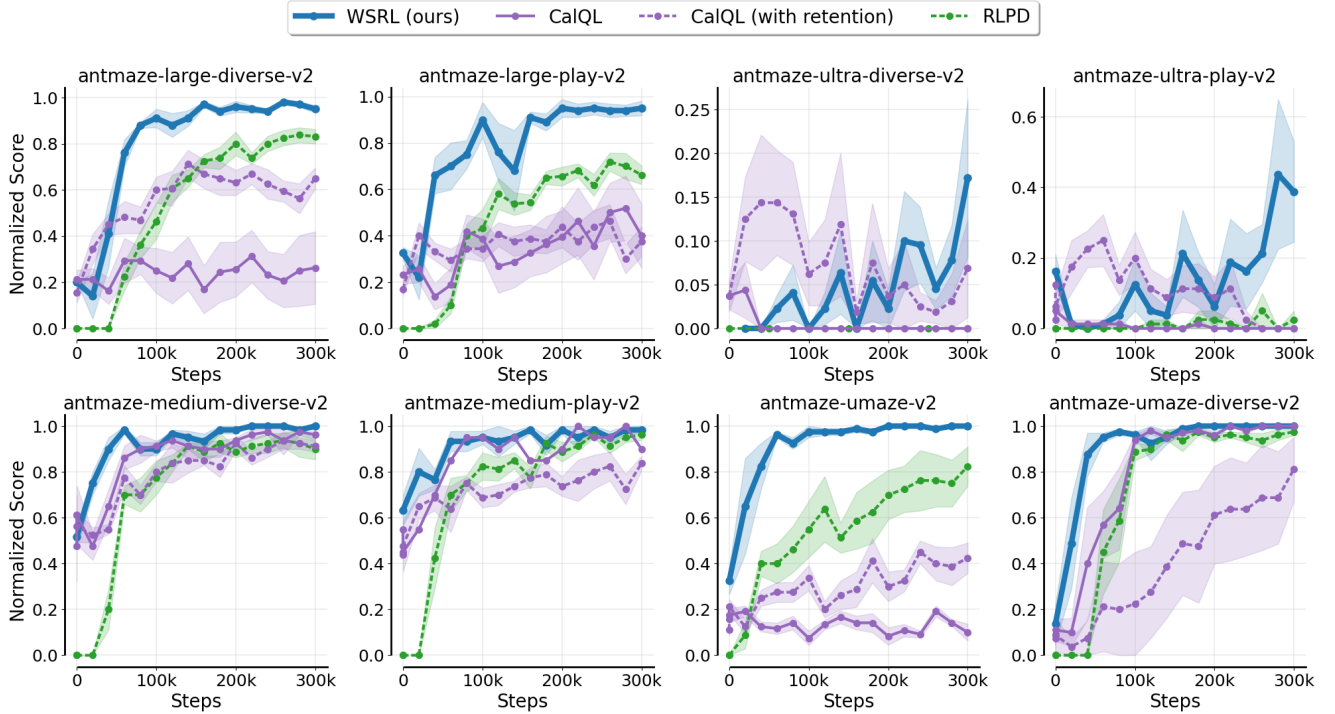


图 16: WSRL 在所有八个 D4RL antmaze 环境上的表现，以及 RLPD 和 CalQL 基线。步骤 0 表示 WSRL 和 CalQL 微调的开始，以及 RLPD 的开始。实线表示不保留离线数据，虚线表示保留离线数据。

B. Mujoco 运动环境上的结果

此外，本文还在无保留微调设置下，将 WSRL 应用于九个不同的 Mujoco 运动领域。具体而言，本文实验了三种不同的机器人形态（Halfcheetah、Hopper 和 Walker），每种形态对应三种不同类型的数据集。随机数据集由随机策略收集；专家数据集由使用 SAC 训练至收敛的策略收集；中等回放数据集由训练至性能约为专家 1/3 的策略的回放缓冲区收集。如图 17 所示，WSRL 优于或接近最佳基线方法。

对于 WSRL，超参数与第 5 节及附录 I 中列出的完全一致，仅有一处例外：其预训练策略和价值函数使用 CQL 离线训练而非 CalQL。这是因为这些离线数据集具有密集奖励且不以终止状态结束，因此没有真实回报来支持 CalQL 正则化器。出于同样原因，图 17 中未包含 CalQL 基线。图 17 中的 IQL 和 CQL 基线均不保留离线

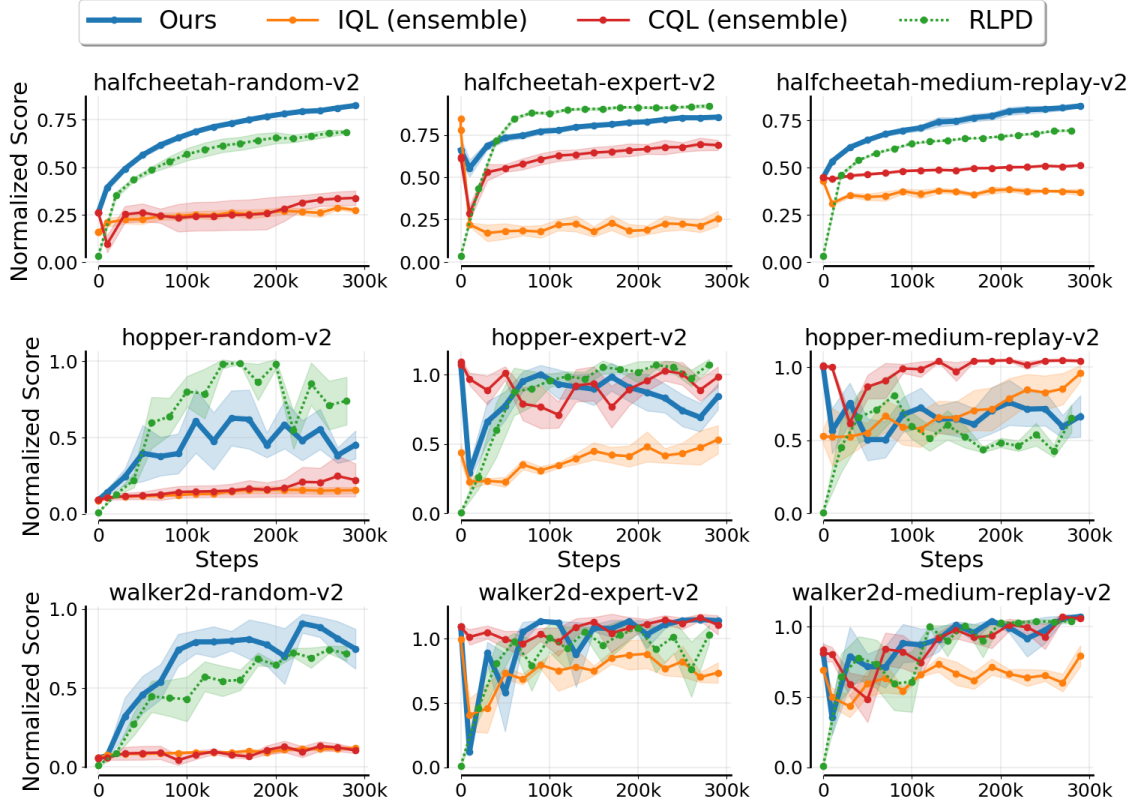


图 17: WSRL 在九个具有密集奖励的 Mujoco 运动环境上的表现, 以及与基线的对比。步骤 0 表示 WSRL 和 CalQL 微调的开始, 以及 RLPD 的开始。实线表示不保留离线数据, 虚线表示保留离线数据。

数据, 并使用 10 个 Q 函数的集成, 同时在 Q 函数中使用层归一化。RLPD 确实保留离线数据。

C. 实验设置

(1) D4RL 中的 Antmaze 任务 [12] 是一类长时程导航任务, 要求控制一个 8 自由度的蚂蚁机器人以稀疏奖励到达目标。智能体需要学会从次优数据集中“拼接”经验。除了 D4RL 中的原始迷宫, 我们还加入了 Antmaze-Ultra [25], 一个更大且更具挑战性的迷宫。我们在第 5 节中仅包含三个最困难的 Antmaze 环境, 并在附录 A 中提供所有八个 Antmaze 的结果。(2) Kitchen 环境是一个长时程操作任务, 控制一个 9 自由度的 Franka 机械臂在模拟厨房中依次完成 4 个子任务。(3) Adroit 环境是一组灵巧操作任务, 控制一个 28 自由度的五指手将笔操作到目标位置、开门以及将球移动到目标位置。智能体在成功时观察到二元奖励。每个数据都有一个离线数据集, 提供 25 条人类演示的狭窄离线数据集以及由行为克隆策略收集的额外轨迹。(4) D4RL 中的 Mujoco 运动环境是密集奖励设置, 智能体学习控制机器人关节以执行各种运动任务。

D. 预热阶段的消融研究

不同预热类型的影响。一个自然的问题随之而来：为什么预热回放缓冲区的简单方法能在微调期间显著提升性能？一种假设是，用一些数据初始化回放缓冲区有助于防止早期过拟合，正如在线强化学习中的许多工作所发现的那样[38]。为了检验这一假设，我们在图 18 中绘制了用随机动作初始化的微调性能，并将其与用预训练策略动作初始化以及完全不初始化缓冲区进行比较。很明显，用随机动作初始化缓冲区显著不如预热方法，实际上与完全不初始化缓冲区相比也没有提供太多好处。这表明预热阶段起作用的原因并非防止过拟合。

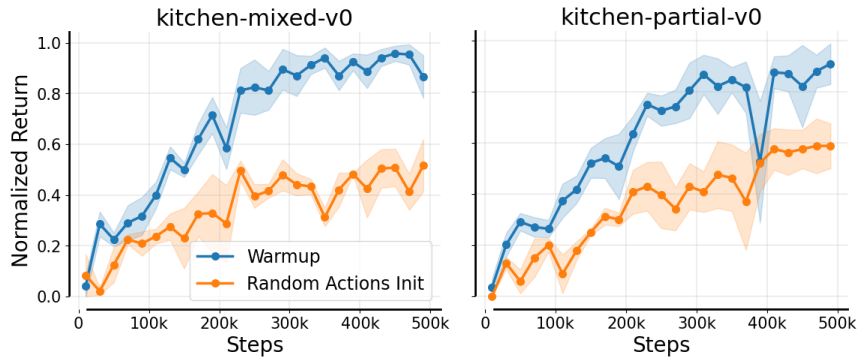


图 18: 比较用随机动作和预训练策略动作初始化缓冲区：在 kitchen-mixed（左）和 kitchen-partial（右）上，用预训练策略动作初始化效果显著更好。

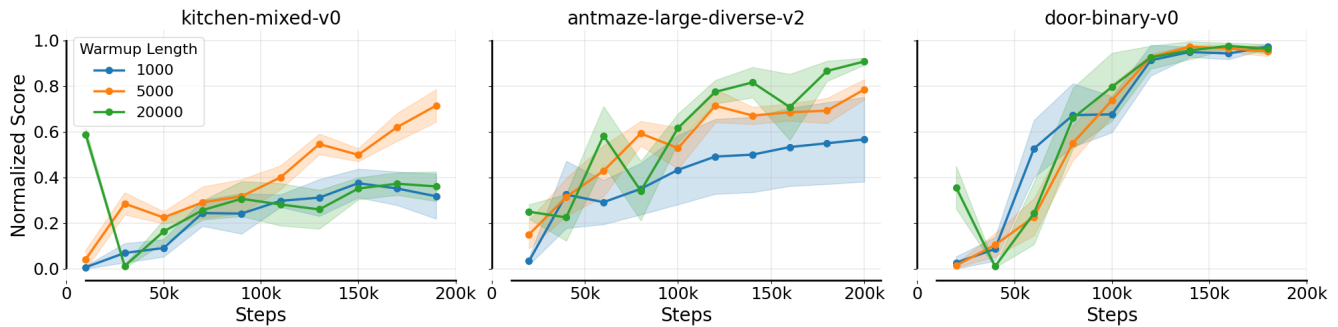


图 19: 长度为 1k、5k、20k 的预热阶段对 Kitchen-mixed（左）、Antmaze-large-diverse（中）和 Door-binary（右）的影响。

不同长度预热的影响。由于预热阶段似乎对高效的在线微调至关重要，我们研究了该预热阶段的长度是否会影响微调性能。图 19 展示了在三个不同环境中长度为 1k、5k 和 20k 的预热阶段。很明显，较短的预热阶段（1k）有时会导致较差的渐近性能或微调过程中的不稳定性。另一方面，较长的预热阶段也可能有害（例如在 **Kitchen-mixed** 环境中），因为它向回放缓冲区添加了过多类似离线数据，从而减慢了在线改进速度。在 WSRL 中，我们未对图 19 所示之外的预热阶段长度进行调整，并且对所有环境使用 5000 个预热步骤。

E. 微调开始时的遗忘与恢复：一项分析

为了进一步说明离线到在线强化学习智能体在在线微调开始时的行为，我们在

图 20 中展示了 WSRL 以及另外两种算法的性能，其评估间隔远小于图 7。我们在六个不同环境中对所有智能体进行 50,000 步在线微调，并每 2,000 个环境步骤评估一次。

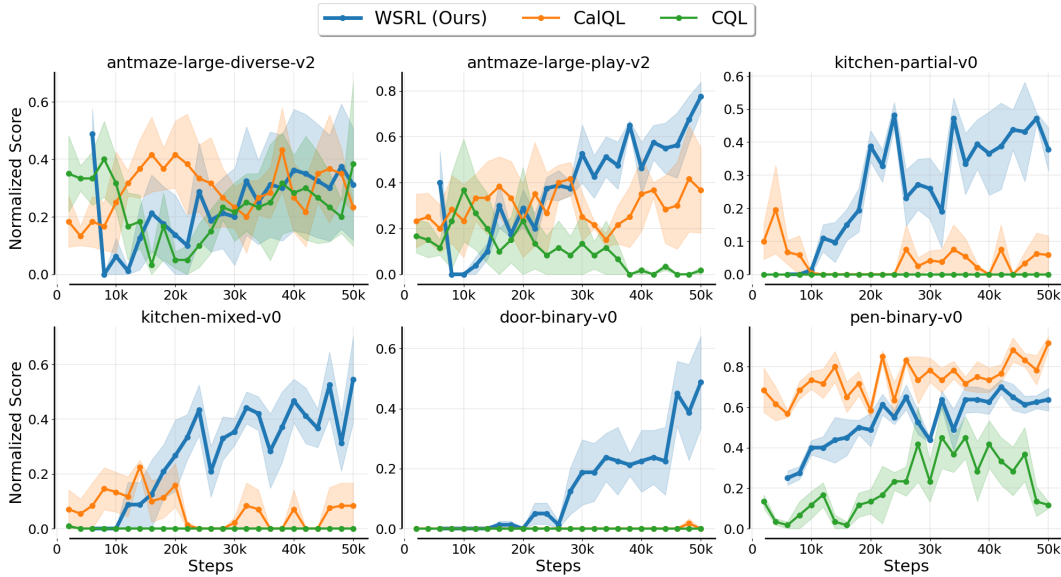


图 20: 使用更密集评估间隔的微调前 50,000 步。图中的第 0 步表示在线微调的开始。WSRL 在预热 $K = 5000$ 步后开始被评估。所有智能体均在无保留微调设置下进行评估。

图 20 显示，WSRL 在预热 5,000 步后策略性能出现初始下降，但恢复速度远快于 CQL 和 CalQL。我们假设，在无离线数据保留设置下，在线微调开始时出现这种下降可能是不可避免的，因为策略正在经历与其训练时不同的状态，甚至可能是从未见过的状态（我们将在下文对此进行更详细的分析）。此外，在某些环境中（例如二元奖励环境），策略的微小波动可能会表现为实际策略性能的巨大变化。然而，这种短暂的性能下降并不意味着策略/Q 函数已被灾难性地破坏，WSRL 比同类算法恢复得更快，并且比 RLDP 等在线强化学习算法学习得更快（见图 9）这一事实证明了这一点。如果这种初始下降会破坏策略中的所有预训练知识，那么我们就不会期望快速恢复。

事实上，一般而言，不可能构建一种无数据保留的微调算法，使其在从离线数据转向在线训练时，在所有环境和离线数据组合上的性能不会初始下降 [58]。直观上，这是因为这违反了一种“没有免费午餐”的结果：例如，考虑一个稀疏奖励问题，其中奖励函数是动作上的任意非光滑函数，此时即使策略动作的微小变化也会导致回报的灾难性变化。因此，仅根据性能来推断算法是否丢失了先验知识可能不是最有信息量的。相反，衡量灾难性遗忘的一个更有意义的度量是评估无数据保留的微调算法偏离其预训练的程度，以及它适应在线状态-动作分布的速度。

因此，为了研究策略和 Q 函数偏离其离线预训练的程度

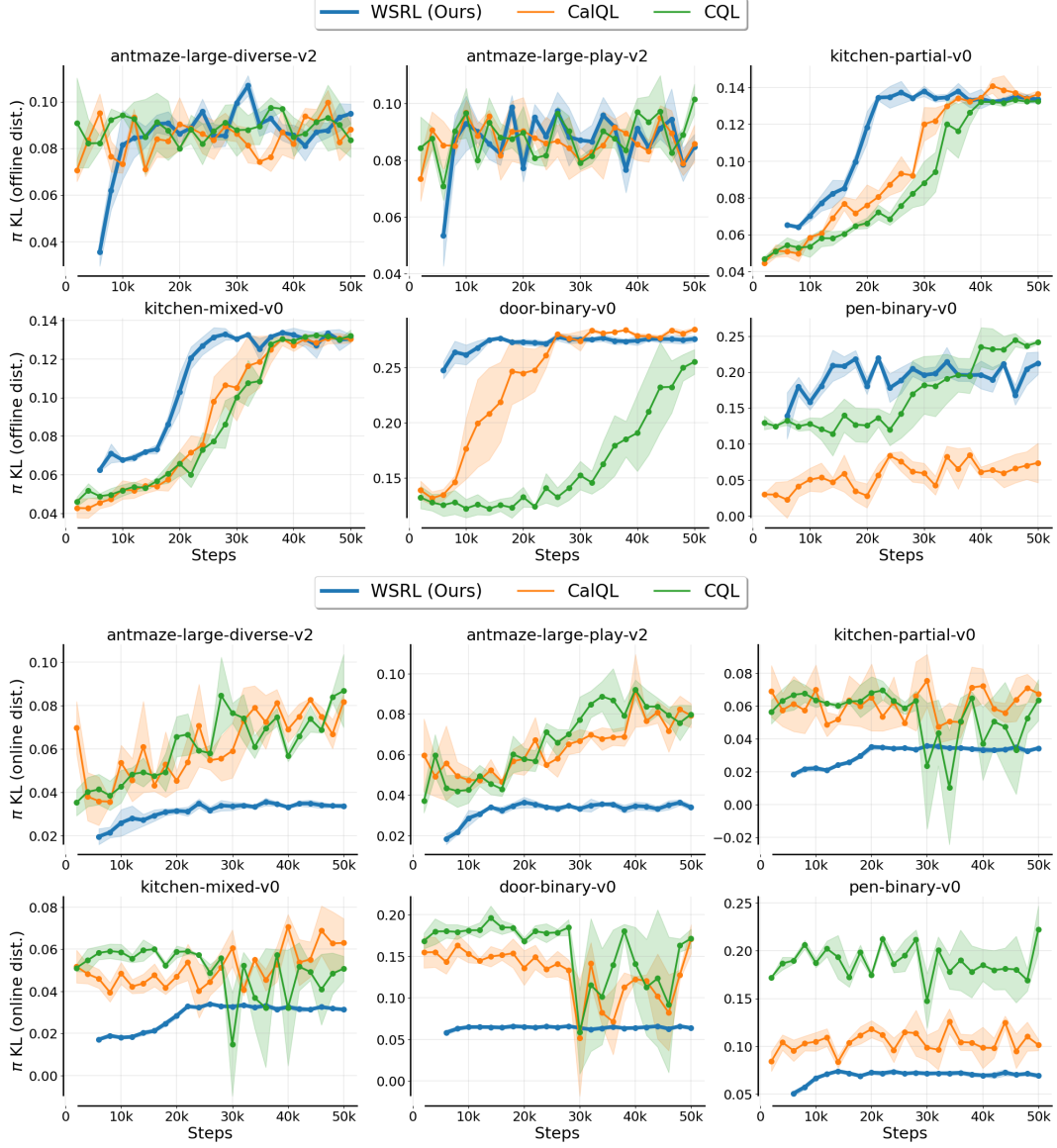


图 21: 在微调的前 50k 步中, 预训练离线策略与微调在线策略之间的策略 KL 散度 $D_{KL}(\pi_{\psi}^{\text{pre}}||\pi_{\psi})$ 。上图显示了在离线数据集分布上评估的 KL 散度; 下图显示了从回放缓冲区采样的在线状态和动作分布上的 KL 散度。WSRL 在预热的前 5,000 步中未绘制。CQL 和 CalQL 不保留离线数据。

在初始“性能下降”阶段, 我们绘制了预训练策略/价值函数与微调后策略/价值函数之间的 KL 散度。图 21 展示了策略 $D_{KL}(\pi_{\psi}^{\text{pre}}||\pi_{\psi})$ 在在线分布和离线分布上评估的 KL 散度。在图 21 (上) 中, 我们可以看到 $D_{KL}(\pi_{\psi}^{\text{pre}}||\pi_{\psi})$ 在离线分布上对于所有三个智能体在微调过程中普遍增加。这一增加表明微调后的策略在数据集分布的至少某些部分上偏离了预训练策略。这在无保留微调设置中实际上是预期的, 因为存在从离线到在线的分布偏移。更具体地说, 例如在 Antmaze 环境中, 离线数据集表现出非常多样的状态-动作分布, 几乎覆盖了整个迷宫中的所有位置, 而微调是一个单目标导航任务。在无保留微调中, 智能体被激励忘记离线数据集中与微调任务无关的部分, 并专门针对在线任务。

与 CQL 和 CalQL 相比, WSRL 通常具有相同的 D_{KL} 渐近值, 但收敛速度更快。这表明 WSRL 相比其无保留对应方法, 能够更快地主动适应在线分布, 这或许得益于其仅在在线数据上优化的非保守目标以及在线强化学习期间的高更新数据比。

另一方面, 图 21 (下) 显示 WSRL 在在线分布上的 D_{KL} 略有增加, 但与 CQL 和 CalQL (无数据保留) 相比要小得多。这表明 WSRL 的策略在在线分布上几乎保持不变。这是可取的, 因为预训练策略已经具有良好的性能, 而一个有效的微调算法在针对未见 (但非分布外) 状态进行轻微调整时, 不应忘记这种能力。总之, 由于从离线预训练到无保留微调的分布偏移, WSRL 正在遗忘离线数据集中在离线预训练期间学习到的经验, 但这些经验实际上对于专门针对在线任务而言是无关的。相反, 它正在专门针对在线任务。

除上述对策略的分析外, 在图 22 中, 我们绘制了预训练 Q 函数 Q_{θ}^{pre} 与微调 Q 函数 Q_{θ} 的 KL 散度。我们将 Q 值通过 Softmax 函数归一化为分布, 并绘制 $D_{KL}(\text{softmax}(Q_{\theta}^{\text{pre}}) || \text{softmax}(Q_{\theta}))$ 。我们在从离线数据集分布 (图 22 上) 和在线回放缓冲区分布 (图 22 下) 采样的状态上, 以及从 $\pi_{\psi}^{\text{pre}}(s)$ 与 $\pi_{\psi}(s)$ 等权混合采样的动作上评估 D_{KL} 。结果表明, CQL 和 CalQL 的 Q 函数通常与预训练显著发散, 这可能是由于第 3 节中描述的螺旋下降现象所致。相比之下, WSRL 的 Q 函数在在线和离线分布上均保持稳定, 表明 WSRL 没有遗忘其预训练的 Q 函数。

总之, 上述关于 KL 散度的分析表明, WSRL 在微调过程中保持稳定并快速适应, 且不会遗忘从离线预训练中学到的先验知识。

F. 在微调开始时冻结策略是否有帮助?

由于附录 E 显示微调初期策略性能会出现短暂下降, 一个自然的问题是这种下降是否可以避免。避免策略性能下降最直接的方法是在初始微调期间冻结策略。换句话说, 在微调开始后的 N 步内, 我们仅通过 Q 函数传递梯度并训练 Q 函数, 但冻结策略。经过 N 在线步骤后, 我们开始同时训练策略和 Q 函数。

图 23 展示了冻结策略 $N = \{10k, 30k\}$ 步后 WSRL 的性能。显然, 即使我们冻结策略若干步以使 Q 函数在线调整, 策略在解冻后仍会出现下降。事实上, 这在某种程度上是预期结果, 因为策略需要适应分布外 (OOD) 的在线状态-动作分布以及新的在线 Q 函数, 而这种适应过程预计会使策略性能变差。

G. 为什么预热可以防止 Q 值发散

在 WSRL 中, 策略与价值函数通过 CalQL[37] 进行离线预训练, 在线微调过程则采用 SAC[18]。强化学习算法的这种变更可能导致校准不准问题, 即预训练值比真实值更为悲观。如第 3 节所示, 当通过贝尔曼更新回传悲观的 Q 目标值时, 这会损害微调效果。当贝尔曼目标在分布外状态-动作对上计算时, 这种损害尤为严重, 因为根据悲观预训练的特性, 分布外状态-动作对的值比离线数据集中出现的状态-动作对更为悲观。如果没有预热阶段, 智能体将把分布外数据收集到缓冲区中, 导致

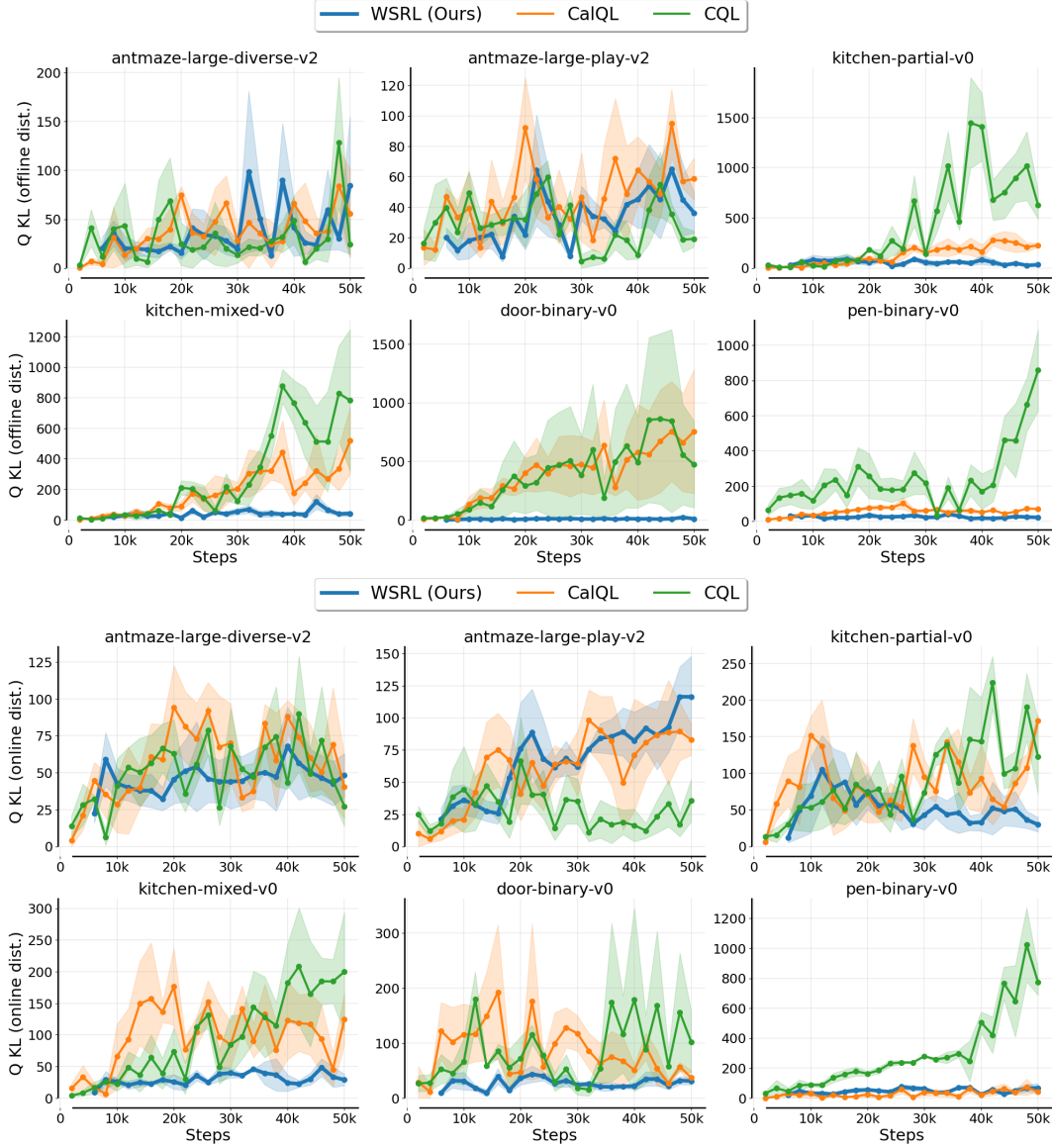


图 22: 在微调的前 50k 步中, 预训练 Q 函数与微调后 Q 函数之间的 Q 函数 KL 散度 $D_{KL}(\text{softmax}(Q_{\theta}^{\text{pre}}) || \text{softmax}(Q_{\theta}))$ 。我们通过从离线数据集分布 (上) 和在线缓冲区分布 (下) 中采样状态来评估 Q 函数, 并通过在采样状态上运行 π_{ψ}^{pre} 和 π_{ψ} 来采样动作。WSRL 在预热的 5,000 步内未绘制。CQL 和 CalQL 不保留离线数据。

导致贝尔曼备份使用悲观的 Q 目标值, 进而引发 Q 值发散。这就是第 3 节中的“螺旋下降”现象。然而, 预热通过将更多类似离线数据放入回放缓冲区来解决此问题, 其中 Q 值不那么悲观, 从而防止在线贝尔曼备份中的螺旋下降, 并利用在线强化学习中的高更新数据比快速重新校准 Q 值。

H. 不同价值初始化类型的消融研究

WSRL 对离线强化学习预训练算法不敏感。此外, 我们发现使用哪种具体的离线强化学习算法来获得预训练 Q 值并不关键。在图 24 中, 我们展示了来自 IQL、CQL 和 CalQL 的 Q 值在三个不同环境中表现同样良好, 尽管

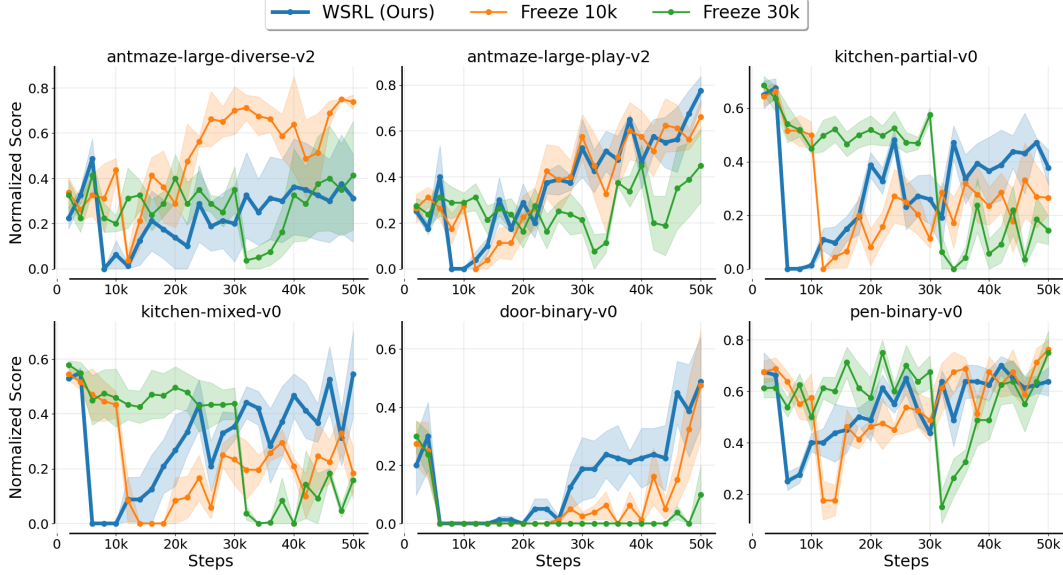


图 23: 在微调开始时冻结策略 $N \in \{10k, 30k\}$ 步并不能防止性能下降。图中展示了 WSRL 与初始策略冻结的 WSRL 在六个环境中的策略性能。步骤 0 是在线微调的开始。

CQL 优化保守的 Q 值，CalQL 保守性较低，而 IQL 完全不保守。特别地，我们观察到以校准 Q 值作为初始化在 Kitchen-mixed 上带来了一些小的性能提升。因此，我们在主要实验中使用来自 CalQL 离线预训练的校准 Q 值。

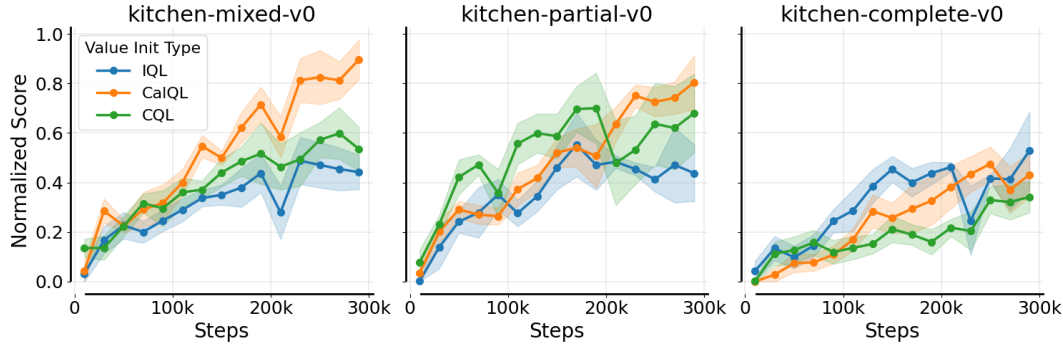


图 24: 用于预训练 Q 值的离线强化学习算法不影响性能：对于 Kitchen-mixed（左）、Kitchen-partial（中）和 Kitchen-complete，WSRL 通过使用来自 CQL、IQL 和 CalQL 的预训练值进行初始化，能够达到相似的性能，尽管 CalQL 初始化在 Kitchen-partial 上有小的优势。

I. 实现细节

WSRL 的代码发布于 <https://github.com/zhouzypaul/wsrl>。

伪代码。 见算法 1。在在线强化学习更新过程中，评论家使用标准时序差分损失进行更新，行动者使用带熵正则化的策略梯度（此处为基于重参数化的策略梯度估计器）进行更新，如软行动者-评论家算法 [18] 中所述。

WSRL 超参数。 我们使用 5K 预热步（ $K = 5,000$ ）。对于 WSRL 中的在线强化学习算法，

Algorithm 1 WSRL: Warm Start Reinforcement Learning**Require:** Offline RL algorithm $\mathcal{A}_{\text{off}}$, Pre-training dataset $\mathcal{D}_{\text{off}}$. $Q_{\theta}^{\text{pre}}, \pi_{\psi}^{\text{pre}} \leftarrow \text{TrainOffline}(\mathcal{A}_{\text{off}}, \mathcal{D}_{\text{off}})$

▷ Offline RL pre-training

Require: $Q_{\theta}^{\text{pre}}, \pi_{\psi}^{\text{pre}}$, Online RL algorithm $\mathcal{A}_{\text{on}}$ with UTD M , Replay buffer $\mathcal{R} \leftarrow \emptyset$, warmup step K . $Q_{\theta} \leftarrow Q_{\theta}^{\text{pre}}, \pi_{\psi} \leftarrow \pi_{\psi}^{\text{pre}}, \mathcal{R} \leftarrow \emptyset$

▷ Initialization

while step $\leq$ max steps **do****if** step $\leq K$ **then** $(s, a, s', r) \leftarrow \text{interact}(\pi_{\psi}^{\text{pre}}, \text{environment})$

▷ Warmup Phase

else $(s, a, s', r) \leftarrow \text{interact}(\pi_{\psi}, \text{environment})$ **end if** $\mathcal{R} \leftarrow \mathcal{R} \cup \{(s, a, s', r)\}$ **if** step $> K$ **then**Batch₁, Batch₂, ..., Batch _{M} $\sim \mathcal{R}$ $Q_{\theta} \leftarrow \text{TemporalDifferenceUpdate}(Q_{\theta}, \text{Batch}_i)$ for M times

▷ High UTD Critic Update

 $\pi_{\psi} \leftarrow \text{PolicyGradient}(\pi_{\psi}, \text{Batch}_1 \cup \dots \cup \text{Batch}_M)$ ▷ Actor Update with Actor Delay of M **end if****end while**

我们在 RLPD [3] 中使用在线 SAC [19] 实现，UTD 为 4，演员延迟为 4（每四个评论家步骤更新一次演员），批大小为 256，演员学习率为 $1e-4$ ，评论家学习率为 $3e-4$ ，温度学习率为 $1e-4$ 。我们使用 10 个 Q 函数的集成，并通过随机子采样 2 个 Q 函数并取最小值来预测 Q 值 [6]。当我们从离线 RL 预训练初始化策略网络和 Q 函数网络时，我们保留这些网络的优化器状态。

在 antmaze 环境中，我们发现通过取 10 个 Q 函数集成上的 TD 目标的最大值来计算 TD 目标很重要，如 Kumar 等人 [29] 所做的那样。这种设计不影响其他环境上的性能。我们假设这是因为 antmaze 环境需要更多的在线乐观性，与 Ball 等人 [3] 的设计决策相配合，如下所述。

环境细节。 所有环境使用折扣因子 0.99。我们按照先前工作 [37, 28, 29] 设置基准任务中的奖励缩放。在 Antmaze 和 Adroit 环境中，我们在目标处使用稀疏奖励 5，每一步使用 -5。在 Kitchen 中，我们在目标处使用 0，对于当前时间步未完成的每个子任务使用 -1，给出可能的奖励值 -4, -3, -2, -1, 0²。在 Mujoco 运动环境中，我们使用原始环境的密集奖励。我们在 Antmaze 上预训练 1M 步，在 Adroit 上预训练 20k 步，在 Kitchen 和 Mujoco 运动上预训练 250k 步。

基线超参数。 所有基于 SAC 的方法使用与上述 WSRL 相同的学习率，基于 IQL 的方法使用演员和评论家学习率 $3e-4$ 。所有具有高更新数据比或 Q 函数集成的方法使用层归一化作为正则化。方法特定的细节如下，超参数取自原始论文，未进一步调整。

IQL。 Antmaze 环境使用期望分位数 0.9，所有其他环境使用期望分位数 0.7。对于演员中的逆温度，我们在 Antmaze 中使用 10，在 Mujoco 运动中使用 3，在 Kitchen 和 Adroit 中使用 0.5。除非另有说明，IQL 使用两个 Q 函数并取它们的最小值来估计

² 在图 7 和图 9 的主要性能比较结果中，厨房环境使用 D4RL 默认的最大回合长度 280，而其他消融和分析实验可能使用 Nakamoto 等人 [37] 的厨房版本，该版本最大回合长度为 1000，但无其他更改。所有绘制的比较均在同一厨房版本上进行。

Q 值。

CQL / CalQL。在 Antmaze 环境中，我们使用 CQL 目标的对偶版本，并将间隙设置为 0.8。在 Mujoco 运动和厨房环境中，我们将 CQL 正则化器权重设置为 $\alpha = 5$ 。在 Adroit 环境中，我们设置 CQL 和 CalQL 在预训练和微调中使用相同的 CQL 正则化器。在保留离线数据的实验中，每个更新批次在 Antmaze、Adroit 和 Mujoco 运动环境中从离线数据中采样 50%，在厨房环境中采样 25% [37]。除非另有说明，CQL/CalQL 使用两个 Q 函数并取它们的最小值来估计 Q 值。

RLPD / SAC (快速)。我们使用 10 个 Q 函数的集成和 UTD 为 4，批大小为 256。遵循 [3]，在 Antmaze 环境中，我们通过随机子采样 1 个 Q 函数来从集成中预测 Q 值。这在 Antmaze 环境中需要更多的在线乐观性。在其他环境中，我们子采样 2 个 Q 函数并取它们的最小值来估计 Q 值。

JSRL。与 RLPD 相同的超参数，我们通过 Q 集成和 UTD 提高了 JSRL 的竞争力。对于每个在线交互回合，JSRL 以概率 0.5 决定是使用预训练的冻结策略进行滚动，还是使用微调策略进行展开。该概率在前 $100k$ 个微调步骤中线性递减至 0。在 JSRL 决定使用预训练的冻结策略进行滚动的回合中，其滚动的步数遵循几何分布，参数为 $\gamma = 0.99$ ，之后微调策略展开直到回合结束。

SO2。与 RLPD 相同的超参数。与原论文不同，我们使用 10 个 Q 集成，以便与具有相同 Q 集成数量的 WSRL 和其他基线进行公平比较。对于动作噪声，我们使用方差为 0.3 的标准正态分布，并将动作噪声裁剪在

$(-0.6, 0.6)$ 。

J. Franka 机器人实现细节

机器人设置。我们使用 SERL 软件套件 [31] 在 Franka 机械臂上运行实验。在在线强化学习微调期间，SERL 将数据收集和智能体更新作为异步进程运行。对于插孔任务，我们将机器人夹爪固定为始终闭合。遵循 Luo 等人 [31, 32] 的做法，我们将机器人末端执行器的位置限制在如下所述的特定边界框内。我们的策略使用预训练的 resnet 编码器，并优化由二分类器提供的稀疏奖励，该分类器在手动收集的 200 个正样本和 500 个负样本上进行了预训练。

数据收集。我们在运行 SERL 时记录回放缓冲区，该 SERL 初始时包含 20 个预收集的专家演示，并使用人工干预来引导和加速在线强化学习 [32]，以实现快速数据收集（当不使用人工干预时，我们发现 SERL 需要超过一小时才能解决插孔任务，这与 Luo 等人 [31] 报告的结果一致）。

环境细节。遵循 Luo 等人 [31, 32] 的做法，我们将机器人末端执行器的位置限制在以插入位置为中心的特定区域内。在为离线数据集收集数据时，我们放宽限制，并使用一个更大的边界框，该边界框张成绿色框的周边，以收集多样化的数据。我们使用折扣因子 0.98，并在边界框内使用随机重置进行在线微调。

超参数。我们采用 5000 步预热，并遵循 SERL [31] 的 SAC 实现。对于 RLPD、CalQL 和 WSRL，我们使用 2 块残差网络多层感知机、层归一化，以及批评家与演员比率为 4、批大小为 256、学习率为 $3e-4$ 。我们使用 10 个 Q 函数的集成，并通过随机子采样 2 个并取最小值来预测 Q 值。对于离线 CalQL，我们使用固定的 CQL 正则化权重 $\alpha = 5$ ，并训练 40000 个梯度步。机器人实验代码可在 <https://github.com/> 找到。

zhouzypaul/wsrl-robot.

K. 使用离线数据集中的转移进行预热

在第 5 节中，本文已证明预热阶段对于利用加权软回报学习（WSRL）进行高效微调至关重要。一个值得探讨的问题是，此类预热数据是否可以通过对离线数据集进行采样来收集，而非如 WSRL 中那样与冻结的预训练策略进行在线交互。因此，本文在图 25 中开展了一项消融实验，将 5,000 步的预热阶段替换为通过初始化在线回放缓冲区的方式。5,000 从离线数据集中随机采样的转移，我们称之为“数据集预热”。如图 25 所示，虽然两种方法在 Adroit 环境中表现相似，但 WSRL 在 Kitchen 上略优，在 Antmaze 上则显著更优。这可能是因为随机采样的 5000 个转移与在线微调任务相关性不高，尤其是在 Antmaze 中，数据集具有多样化的状态-动作覆盖（更详细的讨论见附录 E）。当回放缓冲区用相关性较低的数据初始化时，它在防止 Q 值发散（第 3 节）以及重新校准在线 Q 函数和策略方面的效果较差。这或许凸显了我们方法的实用性：尽管无法访问任何离线数据，WSRL 在无保留微调设置下仍能达到与使用离线数据集转移相似或更好的性能。

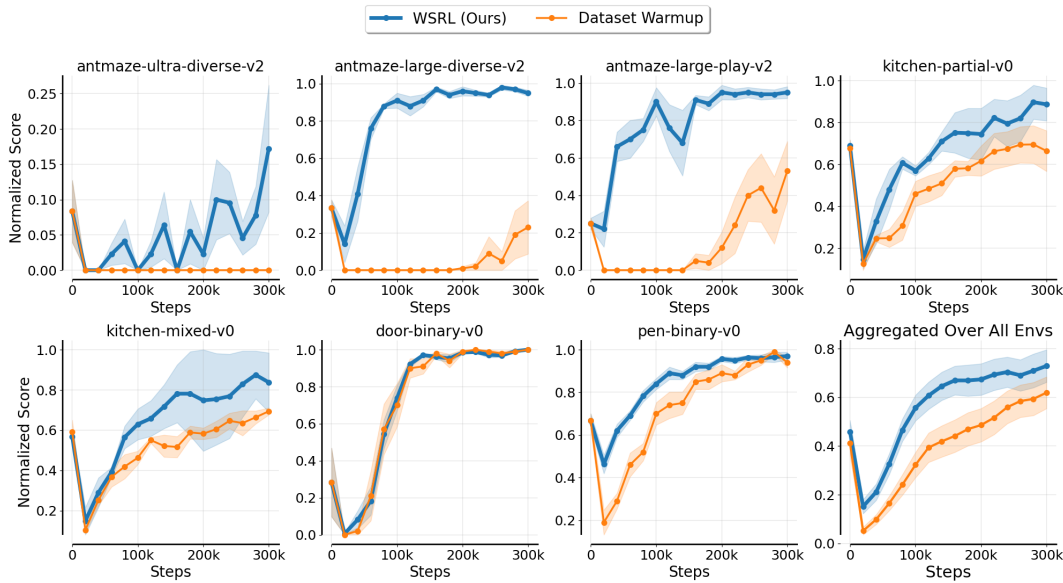


图 25：使用离线数据集中的转移进行预热，其效果不如使用在线交互进行预热（WSRL）。

L. 带离线数据保留的加权稀疏奖励学习（WSRL）

在主论文中，我们已经证明 WSRL 可以在不保留离线预训练数据集的情况下高效微调。一个自然的问题是：如果允许保留离线数据，WSRL 能否做得更好？为了回答这个问题，我们运行 WSRL，并用整个离线数据集初始化在线回放缓冲区。图 26 显示，平均而言，保留离线数据并没有给 WSRL 带来任何优势，可能是因为它已经在离线策略和 Q 函数中具备了必要的知识；实际上，WSRL 在微调后期会稍快一些，这或许是因为它在更多在线数据上更新策略。

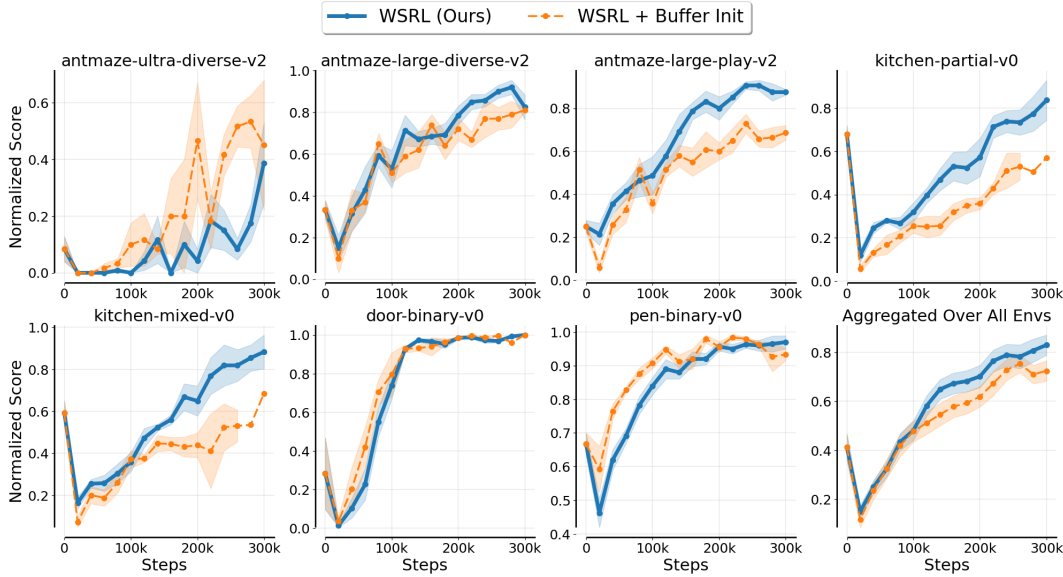


图 26: 用离线数据集初始化回放缓冲区在微调期间并未给 WSRL 带来任何优势, 甚至可能使其稍慢。

M. 消融 Q 集成和层归一化的影响

在 WSRL 中, 我们选择使用最有效的在线强化学习算法, 以高更新数据比 (UTD) 进行微调。遵循 Ball 等人 [3] 的设计选择, 我们还使用了 10 个 Q 函数的集成和层归一化作为正则化, 以在高 UTD 机制下稳定在线训练。在图 7 中, SAC (快速) 和 JSRL 也具有高 UTD, 因此我们也应用了这两种正则化。然而, 我们按照原始论文实现了 IQL、CQL、CalQL, 没有使用这些正则化。为了在无保留微调中消融层归一化和 Q 集成的影响, 我们将两者分别应用于 IQL、CQL 和 CalQL。在图 27 中, 我们在演员和评论家 MLP 的每个密集层之后应用层归一化, 发现它对性能影响极小。在图 28 中, 我们应用层归一化以及 Q 集成, 并将该组合称为 REDQ [6]。我们发现 REDQ 在 Antmaze 任务上有显著帮助, 但在其他环境中增益不大。总体而言, WSRL 仍然显著优于带有额外正则化的 IQL、CQL 和 CalQL。

N. 不同更新数据比下的 WSRL

在第 5 节中, 我们主要对所有方法使用 $UTD=4$ 进行实验。一个有趣的问题是, WSRL 能否从更高的 UTD 中受益, 以及它与具有更高 UTD 的其他快速在线强化学习方法相比如何。在图 29 中, 我们将 WSRL 与 UTD 为 20 的 RLPD 进行比较, 发现虽然 UTD 20 略微提高了性能, 但差异并不大。

O. 不同离线策略水平下的 WSRL

我们研究了当离线预训练策略的专业水平不同时, WSRL 的性能表现。具体来说, 我们考虑了 Kitchen-complete-v0 和 Relocate-binary-v0 这两个对离线强化学习来说特别困难的任务, 在这些任务上使用 CalQL 进行预训练会导致性能不佳。在 Relocate-binary-v0 中, CalQL 完全失败, 预训练性能接近 0; CQL 和 IQL 的预训练性能也为 0, 这表明该任务对离线强化学习智能体来说本质上很难。

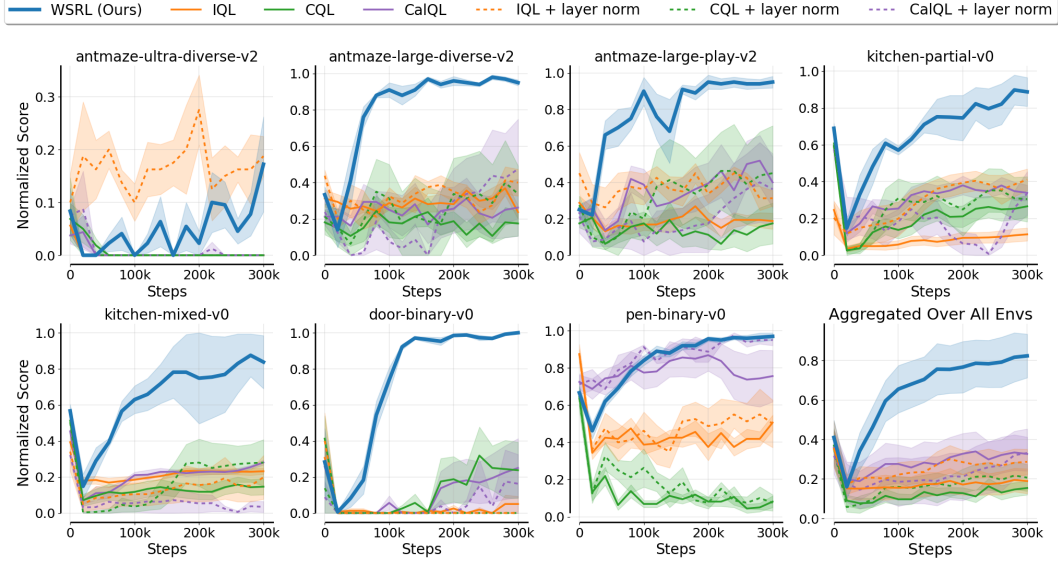


图 27: 层归一化对 IQL、CQL、CalQL 在无保留微调中的性能没有影响。

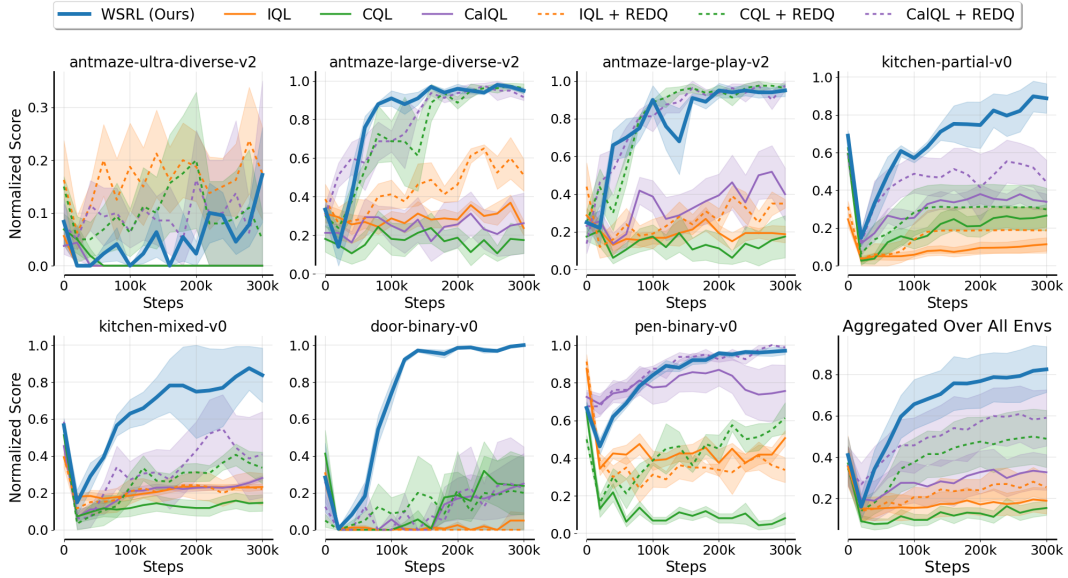


图 28: 层归一化和 Q 集成对 IQL、CQL、CalQL 在无保留微调中的影响: 它在 Antmaze 环境中大有裨益, 但在其他环境中效果不明显。

在 Kitchen-complete-v0 中, 尽管我们做了调参努力, CalQL (15.47%) 的性能仍显著低于 IQL (70.83%), 这表明 CalQL 在该领域学习好的 Q 函数存在一些固有的局限性。不出所料, 图 30 显示 WSRL 也表现不佳: 虽然 WSRL 在 Kitchen-complete-v0 中通过非零初始化可以学到一些东西, 但在 Adroit-binary-v0 中完全无法学习。这是意料之中的, 因为当预训练失败时, 用预训练网络进行初始化可能不会带来任何有用的信息增益, 反而可能通过降低网络的可塑性 ([38]) 而损害微调, 这是在线强化学习中已知的问题。在这种情况下, 人们可能希望采用不同的方法 (例如在线强化学习), 而不是从糟糕的初始化进行微调。

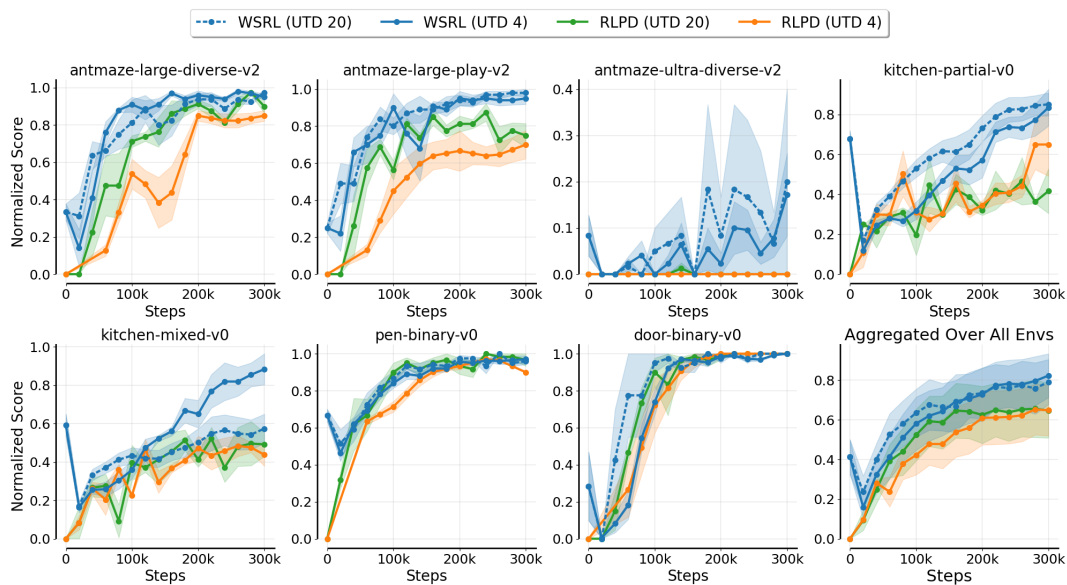


图 29: UTD 20 与 UTD 4: 对于 WSRL 和 RLPD, UTD 20 的性能仅略好于 UTD 4。

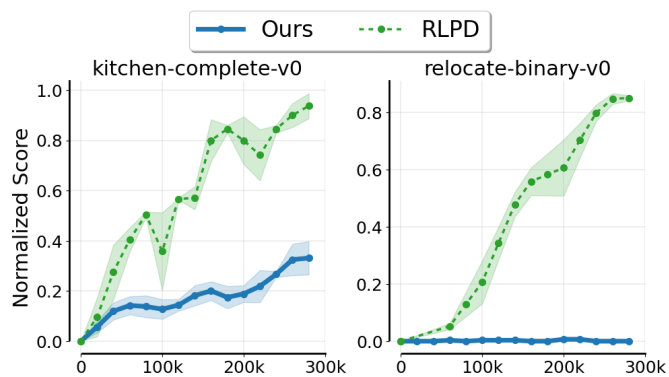


图 30: 在预训练完全失败的环境中, WSRL 表现不佳。